

Phasor Data Concentrator (PDC) Overview

A Phasor Data Concentrator (PDC) is a critical component in modern power systems that aggregates and processes data from multiple Phasor Measurement Units (PMUs). PMUs are devices that measure electrical waves in a power grid, capturing real-time data such as voltage, current, and frequency. The PDC collects this synchronized data from multiple PMUs, ensuring accurate time alignment using GPS timestamps.

How a PDC Works:

1. **Data Collection**: PDCs receive streaming data from multiple PMUs over communication networks using protocols such as IEEE C37.118.
2. **Synchronization**: The PDC time-aligns the data from different PMUs to provide a consistent view of the power system.
3. **Processing & Storage**: The synchronized data is processed for real-time visualization, analysis, and storage in structured databases like MySQL for further evaluation.
4. **Transmission**: The processed data can be sent to control centers for grid monitoring, stability assessment, and fault detection.

Application in the Real-Time Phasor Data Concentrator (RTPDC) System:

The RTPDC system integrates multiple technologies such as **C++**, **MySQL**, **ImGui**, and **IEEE PC37.118** to create a robust and efficient real-time phasor data processing framework. It leverages **socket programming (TCP/IP & UDP)** for real-time data transmission, an **ImGui-based interactive UI** for visualization, and **structured data management using MySQL** to ensure fast queries and reliable transaction handling.

The following sections of this document provide detailed backend and frontend implementation details of the RTPDC system.

Table of Contents

- Introduction to the documentation
 - File structure
 - `Backend.cpp`
 - Modules
 - `socketinit.cpp`
 - `connection.cpp`
 - `cmdframe.cpp`
 - `structures.h`
 - `utilities.cpp`
 - Contribution
-

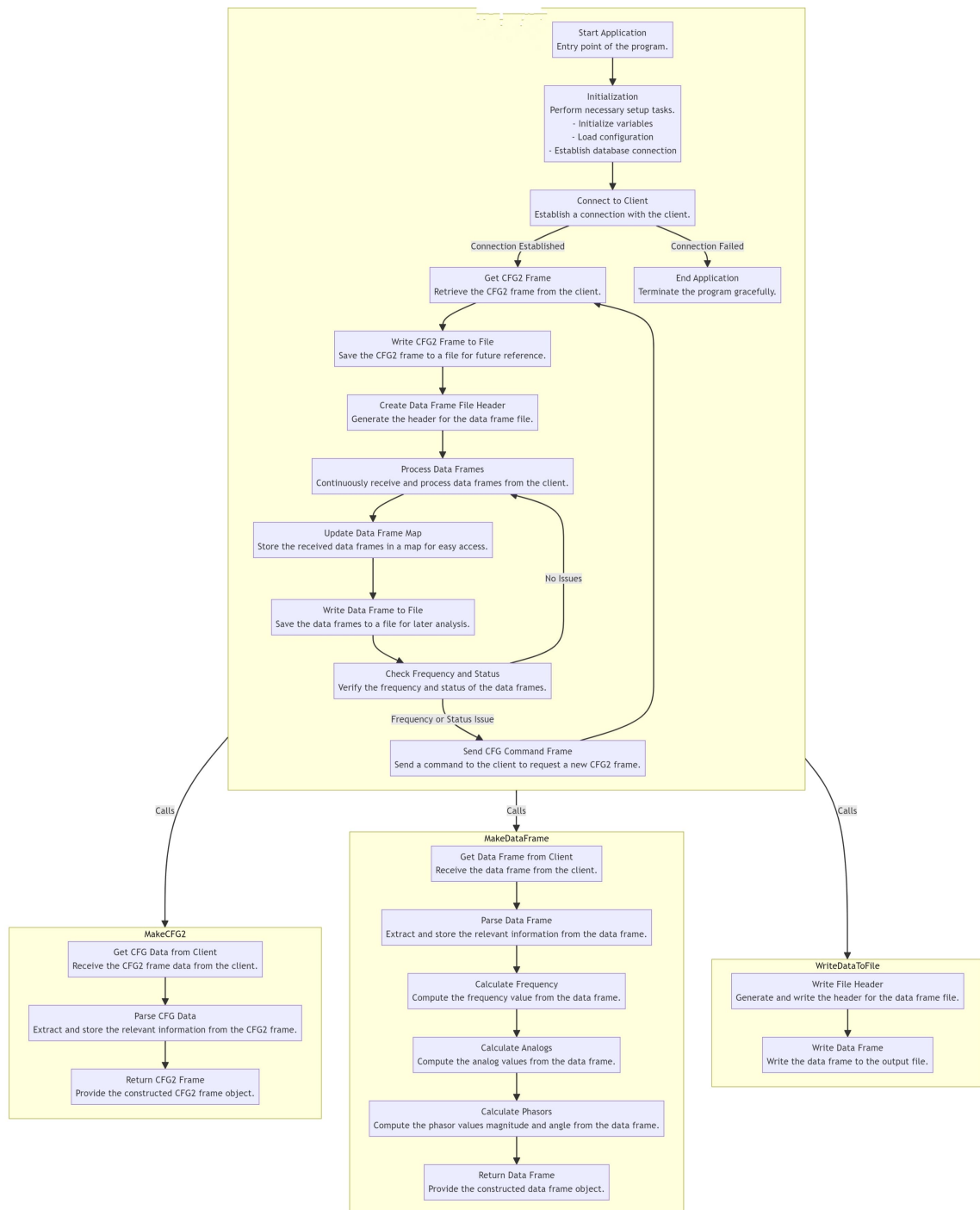
Introduction to the documentation

This document serves as the comprehensive developer's guide for the RtPDC(Real time phasor data concentrator) software, delving into an in-depth exploration of all functions involved in the backend application for the software. This guide is designed to equip developers with the necessary knowledge and understanding to effectively work with the software's codebase.

In the following sections, you will find a detailed description of each function integral to the backend application. These descriptions will facilitate your understanding of their operation, enabling you to build or modify the software according to your needs.

By providing a thorough understanding of the software's backend functions, this documentation aims to empower developers to make the most out of the RtPDC software, leveraging its features and capabilities to their full potential.

Flow chart of the program



File structure

```
|— backend.cpp
|— backend.h
|— modules
|   |— connection.cpp
|   |— structures.h
|   |— cmdframe.cpp
|   |— socketinit.cpp
|   |— utilities.cpp
```

General Instructions

→ To run only the backend program create a `main.cpp` in the top level directory(same directory as of backend.h)

→ copy the following code

```
#include "../backend.h"
int main()
{
    bool isClose = false;
    //change the pmu ip,port and id
    PMU pmu1("172.26.36.97", 4712, 1);
    PMU pmu2("172.26.36.98", 4722, 2);
    std::vector<PMU> pmuVector =
    {
        pmu1,
        pmu2
    };
    dbCred db
    (
        "localhost",
        "your_db_username",
        "your_db_password",
        3305
    );
```

```
std::unordered_map<int, data_frame> dfMap;  
std::vector<std::pair<SOCKET, int>> sockets;  
std::unordered_map<int, cfg2_frame> cfgMap;  
backend  
(  
  isClose,  
  1,  
  pmuVector,  
  1,  
  db,  
  dfMap,  
  sockets,  
  cfgMap  
);  
return 0;  
}
```

→ Compile and run with Visual studio using mysql and ws2_32 libraries.

Codebase

Backend.cpp

This is the main file of the backend operations of the RtPDC software. It includes all the modules.

This file does not have an entry point for the main application but has a function named *backend* that updates required objects passed by reference. Please find the descriptions of all the functions present in this file.

backend

```
void backend
(
    bool& isClose,
    int numThreads,
    std::vector<PMU> pmuVector,
    bool csvOrDb,
    dbCred dbCred,
    std::unordered_map<int,
    data_frame>dfMap,
    std::vector<std::pair<SOCKET,
    int>>& sockets,
    std::unordered_map<int,
    cfg2_frame>& cfgMap
)
```

The function accepts the following arguments

- `bool& isClose` : This is a reference to a boolean variable. It's used to indicate whether the communication with the PMUs should close or not. It is updated by the frontend software depending on the user actions to keep running the application or close it.
- `int numThreads` : This integer parameter specifies the number of threads the backend should use. It basically represents the number of PMUs the user wants to connect to and creates corresponding number of threads.
- `std::vector<PMU> pmuVector` : This is a vector of `PMU` objects. `PMU` is oops representation of the physical PMU which defined more clearly in

"structures.h" module.

- `bool csvOrDb` : This boolean parameter indicates whether the backend should use a CSV file or a database for storing or retrieving data. It's a flag to switch between two storage options.
- `dbCred dbCred` : This parameter is of type `dbCred`, which holds database credentials. It's used to authenticate and connect to a database if `csvOrDb` is set to "1" and use the MySQL server.
- `std::unordered_map<int, data_frame> dfMap` : This is an unordered map that maps integers to `data_frame` objects. `data_frame` represents a data structure for storing data frames as specified by the IEEE PC37.118.2/D4.0 protocol.
- `std::vector<std::pair<SOCKET, int>>& sockets` : This is a reference to a vector of pairs, where each pair contains a `SOCKET` and an integer which is the pmu id. `SOCKET` stores integer values corresponding to the socket and is 1 if the socket is not succesful for communication else it is any other integer. This vector is used for managing network connections and and refer each pmu connection.
- `std::unordered_map<int, cfg2_frame>& cfgMap` : This is a reference to an unordered map that maps pmu ids to their corresponding `cfg2_frame` objects. `cfg2_frame` represents the configuration frame 2 frame as specified in the IEEE docs, This map is used for storing or managing configurations for further parsing of the dataframes.

connectPMU

```
void connectPMU
(
    std::string ip,
    int port,
    int pmuId,
    bool& isClose,
    std::vector<data_frame>& dfBuffer,
    std::mutex& mtx,
    std::unordered_map<int,
    data_frame>& dfMap,
    std::vector<std::pair<SOCKET,
    int>>& sockets,
```

```
std::unordered_map<int,
cfg2_frame>& cfgMap
)
```

The `connectPMU` function is designed to establish a connection to a Phasor Measurement Unit (PMU) over a network, manage the connection state, and handle data frames and configuration frames for the PMU. Here's an explanation of each argument:

- `std::string ip` : The IP address of the PMU to connect to. This is a string representing the network address of the PMU.
- `int port` : The port number on which the PMU is listening. This is an integer value specifying the network port.
- `int pmuId` : The unique identifier of the PMU. This is an integer value used to uniquely identify the PMU in the system. Using these three arguments this function in its implementation creates a socket for tcp communication and adds the socket to the sockets vector.
- `bool& isClose` : A reference to a boolean variable indicating whether the connection should be closed. This flag is updated by the frontend software based on user actions to keep running the application or close it.
- `std::vector<data_frame>& dfBuffer` : A reference to a vector of `data_frame` objects. This vector is used to store data frames received from the PMU. The `data_frame` structure represents a data structure for storing data frames as specified by the IEEE PC37.118.2/D4.0 protocol. This vector stores dataframes from multiple PMUs that come in small intervals defined by "runTimer" function.
- `std::mutex& mtx` : A reference to a `std::mutex` object used for thread-safe access to shared resources, such as the data frame buffer(dfBuffer), sockets vector, cfgMap and so on . Mutexes are used to protect shared data from concurrent access by multiple threads, preventing race conditions and ensuring data integrity . Everywhere in the software this mutex variable is passed by reference to lock and unlock the resources.
- `std::unordered_map<int, data_frame>& dfMap` : A reference to an unordered map mapping PMU IDs to their corresponding `data_frame` objects. This map is used for storing and managing data frames received from the PMUs. The `unordered_map` is a hash table-based container that allows for fast

retrieval of elements based on their keys . This dfMap is used to keep storing the dataframe object corresponding to their ids so as to avoid confusion with multiple PMU dataframes, this dfMap is passed by reference to a function "startDataStream" inside this function.

- `std::vector<std::pair<SOCKET, int>>& sockets` : A reference to a vector of pairs, each containing a `SOCKET` and an integer representing the PMU ID. This vector is used for managing network connections to the PMUs. The `SOCKET` type represents a socket descriptor, which is an integer value used to identify a socket in the system.
- `std::unordered_map<int, cfg2_frame>& cfgMap` : A reference to an unordered map mapping PMU IDs to their corresponding `cfg2_frame` objects. This map is used for storing or managing configurations for further parsing of the data frames. The `cfg2_frame` structure represents the configuration frame 2 frame as specified in the IEEE docs.

This function is designed to be used in a multi-threaded environment, with proper synchronization mechanisms in place to ensure thread safety. It assumes that the provided IP address, port, and PMU ID are valid and that the PMU is reachable and operational.

runTimer

```
void runTimer
(
    std::queue<std::vector<data_frame>>& taskQueue,
    std::vector<data_frame>& dfBuffer,
    bool& isClose,
    std::mutex& mtx,
    std::unordered_map<int, cfg2_frame>& cfgMap
)
```

The `runTimer` function is designed to manage a queue of tasks related to processing data frames from Phasor Measurement Units (PMUs) based on their configuration frames. It calculates the optimal wait time between processing tasks to align with the data rate specified in the configuration frames. Here's an explanation of each argument:

- `std::queue<std::vector<data_frame>>& taskQueue` : A reference to a queue of vectors of `data_frame` objects. This function waits for the optimal time for data frames to come from multiple pmus and then just pushes these dataframes(from dfBuffer) into this taskqueue for further use by a function called "sortQVector".
- `std::vector<data_frame>& dfBuffer` : A reference to a vector of `data_frame` objects. This vector acts as a buffer for storing data frames that are ready to be processed. Once the buffer is full or a certain condition is met, the data frames are moved from the buffer to the task queue for processing.
- `bool& isClose` : A reference to a boolean variable indicating whether the operation should be closed. This flag is checked to determine if the timer loop should continue running or exit.
- `std::mutex& mtx` : A reference to a `std::mutex` object used for thread-safe access to shared resources, such as the task queue and the data frame buffer. Mutexes are used to protect shared data from concurrent access by multiple threads, preventing race conditions and ensuring data integrity.
- `std::unordered_map<int, cfg2_frame>& cfgMap` : A reference to an unordered map mapping PMU IDs to their corresponding `cfg2_frame` objects. This map is used for accessing the configuration frames of the PMUs, which include information such as the data rate at which data frames should be processed.

The function calculates the wait time based on the data rates specified in the configuration frames of each pmu taken from cfgMap, ensuring that tasks are processed at the optimal rate and deciding to drop late packages. It uses a loop to continuously check the `isClose` flag and process tasks until the operation is closed. The function after waiting for the optimal time pushes data from the dfBuffer into the taskQueue.

sortQVector

```
void sortQVector
(
    std::queue<std::vector<data_frame>>& taskQueue,
    bool& isClose,
    MYSQL* dbConnection,
    std::string tableName,
```

```

bool csvOrDb,
std::mutex& mtx,
std::unordered_map<int,
cfg2_frame>& cfgMap
)

```

The `sortQVector` function is designed to process and sort data frames from a queue, then either write them to a CSV file or a database, based on the specified storage option. It also ensures that data frames are processed in the correct order according to their timestamps. Here's an explanation of each argument:

- `std::queue<std::vector<data_frame>>& taskQueue` : A reference to a queue of vectors of `data_frame` objects. This queue holds tasks or vector of data frames in order of their arrival. Each task is represented as a vector of `data_frame` objects, which are popped from the queue for sorting and writing to either a csv or db.
- `bool& isClose` : A reference to a boolean variable indicating whether the operation should be closed. This flag is checked to determine if the processing loop should continue running or exit.
- `MYSQL* dbConnection` : A pointer to a `MYSQL` object representing the database connection. This is used for writing data frames to the database if the `csvOrDb` flag is set to 1 indicating database storage.
- `std::string tableName` : The name of the database table where data frames should be written if the `csvOrDb` flag is set to indicate database storage.
- `bool csvOrDb` : A boolean flag indicating whether to use a CSV file or a database for data storage. A value of `0` indicates CSV storage, while any other value indicates database storage.
- `std::mutex& mtx` : A reference to a `std::mutex` object used for thread-safe access to shared resources, such as the task queue. Mutexes are used to protect shared data from concurrent access by multiple threads, preventing race conditions and ensuring data integrity.
- `std::unordered_map<int, cfg2_frame>& cfgMap` : A reference to an unordered map mapping PMU IDs to their corresponding `cfg2_frame` objects. This map is used for accessing the configuration frames of the PMUs, which include information such as the time base needed for sorting data frames by their timestamps.

The function operates in a loop, continuously checking the `isClose` flag and processing tasks until the operation is closed. It sorts each vector of data frames in the queue based on their timestamps, which are calculated using the time base from the corresponding configuration frame. It then writes the sorted data frames to either a CSV file or a database, depending on the `csvOrDb` flag. The function also ensures that data frames are processed in the correct order by removing any data frames that are older than the last processed data frame from the current vector before processing.

At the end of the function, after processing all data frames in the current vector, it updates the `lastPush` variable with the timestamp of the last processed data frame. This ensures that the next time the function processes data frames, it starts with the data frames that are newer than the last processed data frame, maintaining the correct order of processing.

Database related functions

1. DbSetup

```
MYSQL* DbSetup
(
    const char* DbAddress,
    const char* UserName,
    const char* Password,
    int port
)
```

The `DbSetup` function is designed to establish a connection to a MySQL database, create a database if it doesn't exist, and select it for use. It returns a pointer to the `MYSQL` structure representing the database connection. Here's an explanation of each argument:

- `const char* DbAddress` : A pointer to a constant character string representing the address of the MySQL server. This could be a hostname or an IP address.
- `const char* UserName` : A pointer to a constant character string representing the username to authenticate with the MySQL server.
- `const char* Password` : A pointer to a constant character string representing the password to authenticate with the MySQL server.

- `int port` : An integer representing the port number on which the MySQL server is listening.

The function performs the following steps:

1. **Initialization:** It initializes a `MYSQL` structure using `mysql_init`. This structure is used to hold information about the database connection.
2. **Connection:** It attempts to establish a connection to the MySQL server using `mysql_real_connect`. If the connection fails, it prints an error message, closes the connection (if it was partially established), and returns `NULL`.
3. **Database Creation:** It executes a SQL query to create a database named `RPDC` if it doesn't already exist. If the database creation fails, it prints an error message, closes the connection, and returns `NULL`.
4. **Database Selection:** It executes a SQL query to select the `RPDC` database for use. If the database selection fails, it prints an error message, closes the connection, and returns `NULL`.

At the end of the function, if all the above steps are successful, it returns the pointer to the `MYSQL` structure (`conn`) representing the established database connection. This pointer can then be used for further database operations, such as executing queries or closing the connection.

The function is designed to be used in a context where a connection to a MySQL database is required, and it abstracts the process of establishing the connection, creating the database if necessary, and selecting it for use. This simplifies the setup process for applications that need to interact with a MySQL database.

2. `tableSetup`

```
std::string tableSetup
(
    MYSQL* dbConnection,
    std::set<std::string> phasorHeadings,
    std::set<std::string> analogHeadings
)
```

The `tableSetup` function is designed to create a new table in a MySQL database if it doesn't already exist, based on the provided headings for

phasor and analog data. It returns the name of the table created. Here's an explanation of each argument:

- `MYSQL* dbConnection` : A pointer to a `MYSQL` structure representing the database connection. This is used to execute the SQL query for creating the table.
- `std::set<std::string> phasorHeadings` : A set of strings representing the headings for phasor data. Each heading is used to dynamically add columns to the table for storing phasor data.
- `std::set<std::string> analogHeadings` : A set of strings representing the headings for analog data. Each heading is used to dynamically add columns to the table for storing analog data.

The function performs the following steps:

1. **Generate Table Name:** It calls a function `tableNameFunc()` to generate a name for the new table. This name is stored in the `tableName` variable.
2. **Prepare SQL Query:** It constructs a SQL query string to create a new table with the specified name. The table includes columns for `framesize`, `idcode`, `soc`, `fracsec`, `freq`, and `dfreq`. It also dynamically adds columns for phasor and analog data based on the provided headings. For phasor data, it adds two columns for each heading: one for magnitude and one for angle. For analog data, it adds a single column for each heading.
3. **Execute SQL Query:** It executes the constructed SQL query using the provided database connection. If the table creation fails, it prints an error message, closes the database connection, and returns "NULL".
4. **Return Table Name:** If the table creation is successful, it returns the name of the created table.

This function is useful in scenarios where the structure of the data to be stored in a database is not known in advance or can change dynamically. By dynamically generating the table structure based on the provided headings, it allows for flexibility in handling different types of data. The use of `std::set` for the headings ensures that each heading is unique, preventing the creation of duplicate columns in the table.

3. `writeDataframe2Db`

```

void writeDataframe2Db
(
    MYSQL* dbConnection,
    data_frame dataframe,
    std::string tableName,
    std::unordered_map<int, cfg2_frame>& cfgMap
)

```

The `writeDataframe2Db` function is designed to insert a `data_frame` object into a MySQL database table. It constructs and executes an SQL `INSERT` query based on the data frame's contents and the configuration frame (`cfg2_frame`) associated with the data frame's ID code. Here's an explanation of each argument:

- `MYSQL* dbConnection` : A pointer to a `MYSQL` structure representing the database connection. This is used to execute the SQL query for inserting the data frame into the database.
- `data_frame dataframe` : A `data_frame` object containing the data to be inserted into the database. This object includes fields for `framesize`, `idcode`, `soc`, `fracsec`, `freq`, `dfreq`, and collections for phasor and analog data.
- `std::string tableName` : The name of the database table where the data frame should be inserted. The table name is created by a function called `tableNameFunc` which returns current timestamp as a string.
- `std::unordered_map<int, cfg2_frame>& cfgMap` : A reference to an unordered map mapping PMU IDs to their corresponding `cfg2_frame` objects. This map is used to access the configuration frame for the data frame's ID code, which includes information such as the number of phasors and analogs and their names.

The function performs the following steps:

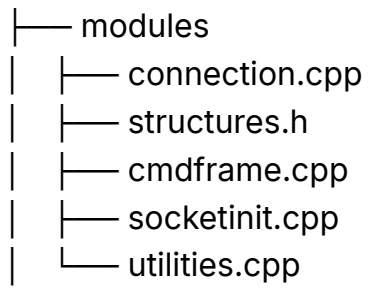
1. **Construct SQL Query**: It begins by constructing the SQL query string for inserting a new row into the specified table. The query includes placeholders for the data frame's fields and for each phasor and analog value.
2. **Dynamically Add Columns**: It dynamically adds columns to the SQL query based on the configuration frame associated with the data

frame's ID code. For each phasor, it adds two columns for magnitude and angle. For each analog, it adds a single column.

3. **Bind Values:** It binds the values from the data frame to the placeholders in the SQL query. This includes the data frame's fields and the values of each phasor and analog.
4. **Execute SQL Query:** It executes the constructed SQL query using the provided database connection. If the insertion fails, it prints an error message.

At the end of the function, if the SQL query is successfully executed, the data frame is inserted into the database table. This allows for the storage of phasor and analog data in a structured format within the database, facilitating further analysis and processing of the data.

Modules



connection.cpp

Index

1. connection
2. startDataStream
3. closeDataStream

Connection.cpp

This is the main file for establishing the connections with the PMUs by creating web sockets.

The description of all the functions present is given below.

Header files

```
#include <winsock2.h>
#include "cmdframe.h"
#include "socketinit.h"
#include "structures.h"
#include "utilities.h"
```

`<winsock2.h>` : Provides access to the Windows Sockets 2 API for network programming.

`"cmdframe.h"` , `"socketinit.h"` , `"structures.h"` , `"utilities.h"` : Custom header files containing function declarations and the custom data structures.

connection

```
SOCKET connection(
    std::string ipAddress,
    int PORT
)
```

- The function takes two parameters:
 1. `ipAddress` : a `std::string` representing the IP address of the PMU to connect.
 2. `PORT` : an `int` representing the port number of the PMU.

The function performs the following steps:

1. The `ipAddress` parameter is a `std::string`, but the `socketInit` function (which is likely a custom function) expects a C-style string (null-terminated character array). So, the `c_str()` method is used to convert the `std::string` to a C-style string.
2. The `socketInit` function(in `socketinit.cpp`) is called with the C-style IP address string and the port number as arguments. This function is responsible for initializing the socket and establishing the connection to the PMU.
3. The function returns a `SOCKET` object, which represents the established connection.

startDataStream

```
void startDataStream(
    SOCKET clientSocket,
    int pmuId,
    bool &isClose,
    std::vector<data_frame> & dfBuffer,
    std::map<int,cfg2_frame>& cfgMap,
    std::mutex &mtx
)
```

This function initiates the data stream from the PMU device.

- `clientSocket` : A `SOCKET` object that represents the connection established with the PMU device.
- `pmuId` : An `int` that represents the ID of the PMU device.
- `isClose` : A reference to a `bool` that checks if the data stream needs to be closed or not.

- `dfBuffer` : A reference to a `std::vector` of `data_frame` objects that store the data frames received from the PMU device.
- `cfgMap` : A reference to a `std::map` that maps the PMU ID to its corresponding configuration frame (`cfg2_frame`).
- `mtx` : A reference to a `std::mutex` used for synchronizing data access.

This function performs the following steps:

- The function initialises an `isCfgAsked` to false.
- The function initiates the data stream by first sending a configuration command frame to the PMU device using the `sendCFGCmdFrame` function (in `cmdframe.cpp`).
- It then creates a configuration frame using the `makeCFG2` function(in `utilities.cpp`).
- Now, the function locks the mutex, updates the `cfgMap` with the PMU ID and its corresponding configuration frame, and then unlocks the mutex.
- A data frame command frame is sent to the PMU device using `sendDFCmdFrame` .
- A file header for the data frame is written using the `writeDataFrameFileHeader` function.
- While the `isClose` variable is `false` , the function creates a `data_frame` using the `makeDF` function, locks the mutex, adds the created `data_frame` to the `dfBuffer` , and unlocks the mutex. This process continues until `isClose` becomes `true` , indicating that the data stream should be closed.

closeDataStream

```
void closeDataStream(
    std::vector<std::pair<SOCKET,int>> &sockets,
    bool & isClose
)
```

This function closes the data stream by closing the Socket.

The function accepts the following arguments:

- `sockets` : A reference to a `std::vector` of `std::pair` objects. Each pair contains a `SOCKET` object and an `int`. The `SOCKET` object represents the connection established with a PMU device and the `int` is the ID of the PMU device.
- `isClose` : A reference to a `bool` that checks if the data stream needs to be closed or not.

The function performs the following steps to close the socket:

- The function takes two input arguments: a `std::vector` of `std::pair` objects (`SOCKET` and `int`), and a reference to a `bool`.
- The function waits for a user input. If the user enters `1`, it sets `isClose` to `1` (true), indicating that the data stream should be closed.
- If `isClose` is true, it loops over the `sockets` vector. For each `SOCKET` object in the vector, it sends a command to close the data stream to the corresponding PMU device by calling the `sendCloseStreamCmdFrame` function (in `cmdframe.cpp`) with the `SOCKET` object and the PMU ID as arguments.
- After this, it closes the `SOCKET` object and calls `WSACleanup()` to terminate use of the Winsock 2 DLL (Dynamically Linked Library).

Structure.h

Index

1. PMU
2. cfg2_frame
3. data_frame

PMU class

```
class PMU
{
public:
    std::string pmuIp;
    int pmuId;
    int port;

    PMU(const std::string &ip, int pt, int id)
    {
        pmuIp = ip;
        pmuId = id;
        port = pt;
    }
}
```

Properties:

- The `PMU` class has three private member variables:
 - `pmuIp`: a string representing the IP address of the PMU
 - `pmuId`: an integer representing the ID of the PMU
 - `port`: an integer representing the port number used by the PMU
- The `PMU` class has a constructor that takes three arguments:
 - `ip`: a string representing the IP address of the PMU
 - `pt`: an integer representing the port number used by the PMU

- `id`: an integer representing the ID of the PMU
- The constructor initializes the member variables `pmuIp`, `pmuId`, and `port` with the values of the `ip`, `pt`, and `id` arguments, respectively.

The `PMU` class represents a Phasor Measurement Unit (PMU) in a power system. It has three private member variables: `pmuIp`, `pmuId`, and `port`. The constructor takes three arguments: `ip`, `pt`, and `id`, which are used to initialize the member variables. The `pmuIp` variable stores the IP address of the PMU, `pmuId` stores the ID of the PMU, and `port` stores the port number used by the PMU. This allows for the creation of `PMU` objects with specific IP addresses, IDs, and port numbers.

cfg2_frame class

```
class cfg2_frame
{
public:
    int framesize;
    int idcode;
    long int soc;
    long int fracsec;
    long int timeBase;
    int numPmu;
    std::string stn;
    std::string format;
    int phnmr;
    int annmr;
    int dgnmr;
    std::vector<std::string> chnam;
    std::vector<std::pair<std::string, long double>> phunit;
    std::vector<std::pair<std::string, long double>> anunit;
    int fnom;
    int dataRate;
};
```

Properties:

1. `int framesize` : Integer representing the frame size.
2. `int idcode` : Integer representing the identification code.
3. `long int soc` : Long integer for the seconds of century.
4. `long int fracsec` : Long integer for the fractional seconds.
5. `long int timeBase` : Long integer for the time base.
6. `int numPmu` : Integer representing the number of PMUs.
7. `std::string stn` : String for the station name.
8. `std::string format` : String for the data format.
9. `int phnmr` : Integer representing the number of phasors.
10. `int annmr` : Integer representing the number of analog values.
11. `int dgnmr` : Integer representing the number of digital status words.
12. `std::vector<std::string> chnam` : Vector of strings for channel names.
13. `std::vector<std::pair<std::string, long double>> phunit` : Vector of pairs for phasor units.
14. `std::vector<std::pair<std::string, long double>> anunit` : Vector of pairs for analog units.
15. `int fnom` : Integer representing the nominal frequency.
16. `int dataRate` : Integer representing the data rate.

This class provides a way to represent and manipulate configuration frames for PMUs. The member variables allow for the storage and retrieval of information about the frame, such as the station name, number of PMUs, and channel units. The vectors of pairs allow for the storage of multiple units for each channel, which can be useful for representing different types of measurements.

data_frame class

```
class data_frame
{
public:
    int framesize;
```

```

    int idcode;
    long int soc;
    long int fracsec;
    std::string stat;
    std::vector<std::pair<double, double>> phasors;
    double freq;
    double dfreq;
    std::vector<double> analogs;
    std::vector<int> digital;
};
#endif

```

Properties:

- `int framesize` : Integer representing the size of the data frame.
- `int idcode` : Integer representing the identification code of the data frame.
- `long int soc` : Long integer representing the seconds of the current week.
- `long int fracsec` : Long integer representing the fractional seconds.
- `std::string stat` : String representing the status of the data frame.
- `std::vector<std::pair<double, double>> phasors` : Vector of pairs of doubles representing the phasor measurements.
- `double freq` : Double representing the frequency.
- `double dfreq` : Double representing the frequency deviation.
- `std::vector<double> analogs` : Vector of doubles representing the analog measurements.
- `std::vector<int> digital` : Vector of integers representing the digital measurements.

This class provides a way to represent and manipulate data frames for PMUs in a power system. The member variables allow for the storage and retrieval of information about the frame, such as the status, phasors, frequency, and analog and digital values. The vectors allow for the storage of multiple values for each channel, which can be useful for representing different types of measurements.

cmdframe.cpp

Index

- a. cmdFrameInit
- b. compute_CRC
- c. int_to_ascii_convertor
- d. long_int_to_ascii_convertor
- e. byte_by_byte_copy
- f. create_command_frame
- g. sendCFGCmdFrame
- h. sendDFCmdFrame
- i. sendCloseStreamCmdFrame

cmdframe.cpp

This is the main file for starting and closing the data sent by PMU.

The description of functions present is given below.

Variable

```
unsigned char DATASYNC[2],  
CFGSYNC[2],  
CMDSYNC[2],  
CMDDATASEND[2],  
CMDDATAOFF[2],  
CMDCFGSEND[2];
```

We initialise six arrays of `unsigned char` data type having a size of 2 `unsigned char` `DATASYNC[2], CFGSYNC[2], CMDSYNC[2], CMDDATASEND[2], CMDDATAOFF[2], CMDCFGSEND[2];` .

cmdFrameInit

```
void cmdFrameInit()
```

The function is used to initialize the values of the global arrays.

For details about setting the values, refer to **IEEE PC37.118.2™/D4.0 Standard for Synchrophasor Data Transfer for Power Systems**

compute_CRC

```
uint16_t compute_CRC(  
    unsigned char *message,  
    int length  
)
```

The `compute_CRC` function calculates the 16-bit Cyclic Redundancy Check (CRC) for a given message.

The function takes in

- `message` (an array of `unsigned char`)
- `length`,

and returns the CRC as an `uint16_t`.

It does the following steps:

- The function initializes the CRC to `0x0ffff`. Then it enters a loop where it computes the CRC using bitwise operations on each byte of the message.
- For each byte `i` in the message, the function does the following:
 1. XORs the most significant byte of the CRC with the `i`th byte of the message.
 2. Shifts the CRC 8 bits to the left.
 3. XORs the CRC with a temporary variable `quick`, which is obtained by XORing `temp` with its right-shifted version and then left-shifting the result twice. This is done to ensure that every bit of the CRC depends on every bit of the input message, making the CRC a very sensitive error detection code.
 4. Returns the final computed CRC.

int_to_ascii_convertor

```
void int_to_ascii_convertor(  
    unsigned int n,  
    unsigned char hex[]  
)
```

The `int_to_ascii_convertor` function converts an integer into an array of ASCII characters.

The function takes in:

- `n` (an unsigned integer)
- `hex` (an array of `unsigned char`)

It does the following steps:

- The most significant byte of `n` is stored in `hex[0]` by performing a right shift operation on `n` by 8 bits.
- The least significant byte of `n` is stored in `hex[1]` by simply assigning `n` to `hex[1]`.

long_int_to_ascii_convertor

```
void long_int_to_ascii_convertor(  
    unsigned long int n,  
    unsigned char hex[]  
)
```

The `long_int_to_ascii_convertor` function converts a long integer into an array of ASCII characters.

The function takes in:

- `n` (an unsigned long integer)
- `hex` (an array of `unsigned char`)

It does the following steps:

- The first (most significant) byte of `n` is stored in `hex[0]` by performing a right shift operation on `n` by 24 bits.

- The second byte of `n` is stored in `hex[1]` by performing a right shift operation on `n` by 16 bits.
- The third byte of `n` is stored in `hex[2]` by performing a right shift operation on `n` by 8 bits.
- The fourth (least significant) byte of `n` is stored in `hex[3]` by simply assigning `n` to `hex[3]`.

byte_by_byte_copy

```
void byte_by_byte_copy(
    unsigned char dst[],
    unsigned char src[],
    int index, int n
)
```

The function `byte_by_byte_copy` is used to copy `n` bytes from the `src` array to the `dst` array, starting at the `index` position in the `dst` array.

The function parameters are:

- `dst[]`: the destination array where the bytes will be copied to.
- `src[]`: the source array from where the bytes will be copied.
- `index`: the starting position in the destination array from where the bytes should be copied.
- `n`: the number of bytes to copy from the source array to the destination array.

The function works as follows:

- It uses a for loop to iterate over `n` elements of the `src` array.
- For each iteration, it copies the `i`th element of the `src` array to the `(index + i)`th position of the `dst` array.

create_command_frame

```
void create_command_frame(
    int type,
    int pmu_id,
```



```
char cmdframe[]  
)
```

The `create_command_frame` function is used to create a command frame.

The function takes in:

- `type` (an int): This represents the type of command to be sent. The options are 1 (Send Configuration), 2 (Send Data On), and 3 (Turn Off Data Transmission).
- `pmu_id` (an int): This represents the ID of the PMU (Phasor Measurement Unit) to which the command is to be sent.
- `cmdframe` (a char array): This is the command frame that will be populated by the function.

The function does the following:

- The function first initializes several global arrays using the `cmdFrameInit` function.
- Then, it converts the frame size (`f`) and `pmu_id` into ASCII using the `int_to_ascii_convertor` function, storing the results in `fsize` and `pmuid` respectively.
- It also converts the current time (seconds and fractional seconds) into ASCII using the `long_int_to_ascii_convertor` function, storing the results in `soc` and `fracsec`.
- Then, based on the `type` given, it populates the `cmdframe` with the appropriate command following the protocol.
- In each case, it copies the command SYNC, frame size, PMU ID, SOC, FRACSEC, and other commands into the `cmdframe` using the `byte_by_byte_copy` function.
- Finally, it calculates the CRC of the `cmdframe` using the `compute_CRC` function and appends it to the end of the `cmdframe`.

sendCFGCmdFrame

```
void sendCFGCmdFrame(  
SOCKET clientSocket,
```

```
int pmuId
)
```

The `sendCFGCmdFrame` function is used to send a configuration command frame to a PMU (Phasor Measurement Unit) through a client socket.

The function takes in:

- `clientSocket` (a SOCKET): This is the client socket through which the command frame will be sent.
- `pmuId` (an int): This is the ID of the PMU to which the command frame is to be sent.

The function does the following:

- The function first creates a command frame of type 1 (Send Configuration) using the `create_command_frame` function. The created command frame is stored in `cmdframe`.
- Then, it sends the `cmdframe` through the `clientSocket` using the `send` function.
- If the `send` function fails (i.e., if it returns `SOCKET_ERROR`), the function prints an error message, closes the `clientSocket`, and cleans up the Winsock environment by calling `WSACleanup`.

sendDFCmdFrame

```
void sendDFCmdFrame(
    SOCKET clientSocket,
    int pmuId
)
```

The `sendDFCmdFrame` function is used to send a Data On command frame to a Phasor Measurement Unit (PMU) through a client socket.

The function parameters are:

- `clientSocket` (a SOCKET): This represents the client socket through which the command frame will be sent.
- `pmuId` (an int): This represents the ID of the PMU to which the command frame is to be sent.

The function works as follows:

- Initially, it creates a command frame of type 2 (Data On) by calling the `create_command_frame` function. The created command frame is stored in `cmdframe`.
- Then, it attempts to send the `cmdframe` through the `clientSocket` using the `send` function.
- If the `send` function fails (i.e., if it returns `SOCKET_ERROR`), the function prints an error message, closes the `clientSocket`, and cleans up the Winsock environment by calling `WSACleanup`.

sendCloseStreamCmdFrame

```
void sendCloseStreamCmdFrame(
    SOCKET clientSocket,
    int pmuId
)
```

The `sendCloseStreamCmdFrame` function is used to send a Turn Off Data Transmission command frame to a Phasor Measurement Unit (PMU) through a client socket.

The function parameters are:

- `clientSocket` (a SOCKET): This is the client socket through which the command frame will be sent.
- `pmuId` (an int): This is the ID of the PMU to which the command frame is to be sent.

The function works as follows:

- It first creates an array of unsigned char named `cmdframe`, with a size of 18.
- Then, it calls the `create_command_frame` function with parameters 3 (which stands for Turn Off Data Transmission), `pmuId`, and a casted `(char *)cmdframe`. The `create_command_frame` function populates the `cmdframe` array with the appropriate command frame.
- The function then attempts to send the `cmdframe` through the `clientSocket` using the `send` function. If the `send` function fails (i.e., if it returns `SOCKET_ERROR`), the function prints an error message, closes the `clientSocket`, and cleans up the Winsock environment by calling `WSACleanup`.

Socketinit.cpp

Index

1. socketInit

SocketInit

```
SOCKET socketInit  
(  
    const char* ipAddress,  
    u_short portNum  
)
```

The `socketInit` function is designed to initialize a socket connection to a server using the provided IP address and port number. It's a common pattern in network programming, especially in client applications that need to communicate with servers over a network. Let's break down the function and its arguments:

Arguments:

1. `const char* ipAddress`: This is a pointer to a constant character string that represents the IP address of the server to which the client wants to connect. The IP address is passed as a string (e.g., "192.168.1.1") to the function.
2. `u_short portNum`: This is an unsigned short integer representing the port number on the server to which the client wants to connect. Port numbers are used to identify specific processes or services running on a server.

Functionality:

1. **Winsock Initialization**: The function begins by initializing the Winsock library, which is necessary for network programming on Windows. It calls `WSAStartup` with the version of Winsock required (in this case, version 2.2) and a pointer to a `WSADATA` structure that will receive details about the

Winsock implementation. If the initialization fails, the function prints an error message and returns `1`, indicating failure.

2. **Socket Creation:** Next, the function creates a socket using the `socket` function, specifying the address family (`AF_INET` for IPv4), the socket type (`SOCK_STREAM` for TCP), and the protocol (0 to automatically choose the appropriate protocol). If the socket creation fails, it prints an error message, cleans up the Winsock environment with `WSACleanup`, and returns `1`.
3. **Server Address Configuration:** The function then sets up a `sockaddr_in` structure to hold the server's address information. It sets the address family to `AF_INET`, converts the IP address string to a network byte order integer using `inet_addr`, and sets the port number to the network byte order using `htons`.
4. **Connection Attempt:** The function attempts to connect to the server using the `connect` function, passing the socket, the address structure, and the size of the structure. If the connection fails, it retrieves the error code using `WSAGetLastError`, prints an error message, closes the socket with `closesocket`, cleans up the Winsock environment, and returns `1`.
5. **Successful Connection:** If the connection is successful, the function returns the socket descriptor (`clientSocket`), which can then be used for sending and receiving data to/from the server.

Summary:

The `socketInit` function is a comprehensive example of setting up a TCP client socket in a Windows environment. It initializes Winsock, creates a socket, configures the server address, attempts to connect to the server, and handles errors gracefully. The function's arguments allow for flexibility in specifying the server's IP address and port number, making it adaptable to different network configurations.

Utilities.cpp

Index

1. makeDf
2. writeLiveTxt
3. writeDataFrameFileHeader
4. writeDataframe2File
5. makeCFG2
6. IEEE754 formats
7. binToFloatAngle
8. binToFloatMag
9. Other helper functions used

makeDf

```
data_frame makeDF
(
    SOCKET clientSocket,
    cfg2_frame& cfg,
    std::string fileName,
    bool& ifCFGAsked,
    int pmuId,
    std::mutex& mtx,
    std::unordered_map<int, data_frame> &dfMap
)
```

the function accepts the following arguments:

- `SOCKET clientSocket` : This is a socket descriptor for the client connection. It is used to receive data from the client.
- `cfg2_frame& cfg` : This is a reference to a `cfg2_frame` object, which contains configuration frame for how the data should be processed. The function uses this configuration to determine how to interpret the received data.
- `std::string fileName` : This is a string representing the name of a file. It is not directly used within the function but is passed to `writeDataframe2File` which later writes the data received into a text file.

- `bool& ifCFGAsked` : This is a reference to a boolean variable. It is used to track whether a configuration command has been sent to the client. The function sets this variable to `true` if a configuration command is sent.
- `int pmuId` : This is an integer representing the ID of a Phasor measurement unit (PMU). It is used in the function calls `sendCFGCmdFrame` and `makeCFG2` to specify which PMU the command or configuration is for.
- `std::mutex& mtx` : This is a reference to a `std::mutex` object. It is used to synchronize access to shared resources, in this case, the `dfMap` unordered map. The function locks the mutex before updating `dfMap` and unlocks it afterward to ensure thread safety.
- `std::unordered_map<int, data_frame> &dfMap` : This is a reference to an unordered map where the keys are integers (representing PMU IDs) and the values are `data_frame` objects. The function updates this map with the newly constructed `data_frame` object.

The `makeDF` function processes data received from a client socket, constructs a `data_frame` object based on the received data and configuration settings, and updates a map of `data_frame` objects. It also handles sending configuration commands to the client if necessary, ensuring thread safety by using a mutex for map updates.

writeLiveText

```
void writeLiveTxt
(
    data_frame df,
    cfg2_frame cfg,
    int pmuId
)
```

The `writeLiveTxt` function takes three arguments:

- `data_frame df` : A `data_frame` object containing various data fields such as `soc`, `fracsec`, `stat`, `freq`, `phasors`, and `analogs`.
- `cfg2_frame cfg` : A `cfg2_frame` object that holds configuration settings, including channel names (`chnam`) and the number of channels for phasors (`phnmr`) and analogs (`annmr`).
- `int pmuId` : An integer representing the ID of a Phasor measurement unit (PMU).

The function writes the contents of the `data_frame` object to a text file named `data_<pmuId>.txt`. It formats the data with hash lines for separation and includes sections for SOC, FRACSEC, STAT, FREQUENCY, PHASORS, and ANALOGS. For

PHASORS and ANALOGS, it uses the channel names from the `cfg` object. The function does not explicitly close the file stream, which is generally recommended for ensuring data integrity and resource management.

writeDataFrameFileHeader

```
std::string writeDataFrameFileHeader
(
    cfg2_frame cfg,
    int pmuId
)
```

The `writeDataFrameFileHeader` function takes two arguments:

- `cfg2_frame cfg`: A `cfg2_frame` object that holds configuration settings, including channel names (`chnam`) and the number of channels for phasors (`phnmr`) and analogs (`annmr`).
- `int pmuId`: An integer representing the ID of a Phasor measurement unit (PMU).

The function generates a file name for a CSV file that includes the current date and time, the PMU ID, and a fixed prefix. It then opens a file named `sortedData.csv` in append mode and writes a header line to it. The header line includes column names for SOC, FRACSEC, PMU ID, STAT, FREQ, and for each phasor and analog channel, it includes both magnitude and angle (for phasors) or just the value (for analogs). The channel names are taken from the `cfg` object.

The function uses the `std::time` and `std::localtime` functions to get the current time and format it as a string in the format `_YYYYMMDD_HHMM`. This string is appended to the file name prefix to create a unique file name for each run of the program.

After writing the header to `sortedData.csv`, the function closes the file stream and returns the generated file name. This file name can be used later to write data frames to the file.

writeDataframe2File

```
void writeDataframe2File
(
    std::string fileName,
    data_frame dataFrame
)
```

The `writeDataframe2File` function takes two arguments:

- `std::string fileName`: A string representing the name of the file to which the data from the `data_frame` object will be written.

- `data_frame dataframe` : A `data_frame` object containing various data fields such as `soc` , `fracsec` , `idcode` , `stat` , `freq` , `phasors` , and `analogs` .

The function appends the contents of the `data_frame` object to the specified file in a comma-separated format. It starts by opening the file in append mode (`std::ios::app`) to ensure that new data is added to the end of the file without overwriting existing content.

The function constructs a string `headingsData` by concatenating the values of the `data_frame` object's fields, separated by commas. It includes `soc` , `fracsec` , `idcode` , `stat` , `freq` , and iterates over `phasors` and `analogs` to include their values. Each phasor and analog value is added to the string with a comma separator.

After constructing the `headingsData` string, the function writes this string to the file and then closes the file stream. This ensures that the data is safely written to the file and that resources are properly released.

makeCFG2

```
cfg2_frame makeCFG2
(
    SOCKET clientSocket,
    int pmuId
)
```

The `makeCFG2` function takes two arguments:

- `SOCKET clientSocket` : A socket descriptor for the client connection. It is used to receive data from the client.
- `int pmuId` : An integer representing the ID of a Phasor measurement unit (PMU).

The function constructs a `cfg2_frame` object by receiving data from the client socket and processing it. It begins by receiving data into a buffer and checking for errors. If an error occurs during the receive operation, the function returns an empty `cfg2_frame` object.

The function then processes the received data to populate various fields of the `cfg2_frame` object, such

as `framesize` , `idcode` , `soc` , `fracsec` , `timeBase` , `numPmu` , `stn` , `format` , `phnmr` , `annmr` , `dgnmr` , and `chnam` . It also constructs lists of channel names (`chnam`), phasor units (`phunit`), and analog units (`anunit`) based on the received data.

The function calculates the nominal frequency (`fnom`) based on a specific byte in the received data. It then writes the constructed `cfg2_frame` object to a file named `CFG_<pmuId>_<date_time>.txt` using the `writeCFG2File` function. The file name includes the PMU ID and the current date and time.

Finally, the function returns the constructed `cfg2_frame` object. This object contains configuration settings for the PMU, including channel names, units, and other parameters, which can be used by other parts of the program to process data received from the PMU.

writeCFG2File

```
void writeCFG2File
(
    std::string fileName,
    cfg2_frame *cfgFrame
)
```

The `writeCFG2File` function takes two arguments:

- `std::string fileName` : A string representing the name of the file to which the configuration data will be written.
- `cfg2_frame *cfgFrame` : A pointer to a `cfg2_frame` object that contains configuration settings for a phasor Measurement Unit (PMU).

The function writes the contents of the `cfg2_frame` object to the specified file. It starts by opening the file and then writes various configuration settings to it, each separated by a hash line for clarity. These settings

include `framesize`, `idcode`, `soc`, `fracsec`, `timeBase`, `numPmu`, `stn`, `format`, `phnmr`, `annmr`, `dgnmr`, and the lists of channel names (`chnam`), phasor units (`phunit`), and analog units (`anunit`).

The function iterates over the `chnam`, `phunit`, and `anunit` lists to write each item to the file. For `phunit` and `anunit`, it writes both the channel name and the unit value. After writing all the configuration settings, it writes the nominal frequency (`fnom`) and the data rate (`dataRate`) before closing the file.

Functions use for converting floating representation into decimal numbers.

before the description of functions, the following section describes the conversion of different floating representations into readable data:

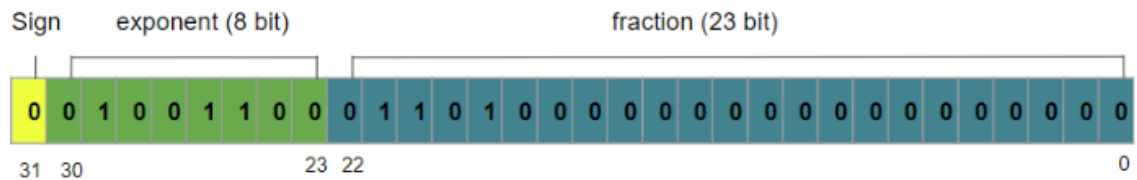
▼ float32

`float32` is the most commonly used of the IEEE754 formats. As suggested by the term "32-bit float", its underlying binary representation is **32 bits long**. These are segmented as follows:

- 1 bit for the **sign** (S)

- 8 bits for the **exponent** (E)
- 23 bits for the **fraction** (F)

When it's stored in memory, the bits look like this:



We can rewrite it as a binary string:

$b_1 b_2 b_3 \dots b_{30} b_{31} b_{32}$

The real value that this 32-bit float stores can be calculated as:

$$(-1)^{b_1} \times 2^{(b_2 \dots b_9)_2 - 127} \times (1.b_{10} \dots b_{32})_2$$

where:

- 127 is called the **exponent bias** and is inherent to the single-precision format
- $\times 2$ means that x must be interpreted as if in base 2 or binary
- $(1.b_{10} \dots b_{32})_2$ means to take the binary bits b_{10} to b_{32} and use it as the fractional part of 1 to form a binary fraction.

Rewriting the formula:

$$(-1)^S \times 2^{E-127} \times (1.F)_2$$

where

- $S = b_1$
- $E = (b_2 \dots b_9)_2$
- $F = (b_{10} \dots b_{32})_2$.

$E=0$ and $F>0$ activates **a separate formula** float32, that formula is

$$(-1)^S \times 2^{-126} \times (0.F)_2$$

There are **other special cases** encoded by specific values for E and F :

E	F	Value
0	0	± 0
0	> 0	$(-1)^S \times 2^{-126} \times (0.F)_2$
$0 < E < 255$	Any	$(-1)^S \times 2^{E-127} \times (1.F)_2$
255	0	$\pm \infty$
255	> 0	NaN

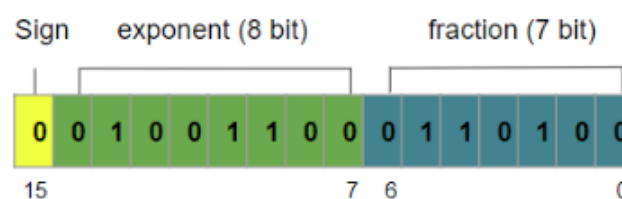
NaN means "**not a number**", which is returned when you divide by zero or perform impossible math using infinity. For the cases of ± 0 and $\pm \infty$, the sign bit determines whether the number is positive or negative.

▼ float16

`float32` is the most commonly used of the IEEE754 formats. As suggested by the term "32-bit float", its underlying binary representation is **32 bits long**. These are segmented as follows:

- 1 bit for the **sign** (S)
- 8 bits for the **exponent** (E)
- 7 bits for the **fraction** (F)

When it's stored in memory, the bits look like this:



Here's a generic formula to calculate the value:

$$(-1)^S \times (1 + F) \times 2^{(E-\text{bias})}$$

Where:

- S is the sign bit (0 for positive, 1 for negative).
- F is the significand represented as a fraction.

- E is the exponent value after subtracting the bias.
- "bias" is the bias value for the exponent.

Bias in floating-point representation adjusts exponent bits to handle positive and negative values uniformly. It's a constant subtracted from the encoded exponent, enabling simpler arithmetic operations. For IEEE 754 single-precision format, the bias is typically 127, ensuring a balanced range of representable exponents around zero.

1) binToFloatAngle

```
double binToFloatAngle
(
    std::string binary,
    int flag
)
```

The `binToFloatAngle` function takes two arguments:

- `std::string binary`: A string representing a binary number.
- `int flag`: An integer flag that determines how the binary number should be interpreted and converted.

The function converts a binary number to a floating-point number representing an angle. The conversion process depends on the value of the `flag` argument:

- If `flag` is `3`, the function interprets the binary string as a 32-bit floating-point number in IEEE 754 format. It extracts the sign, exponent, and fraction parts of the binary number and converts them to a floating-point value. Special cases like NaN (Not a Number), infinity, and zero are handled appropriately. The resulting floating-point value is then converted to an angle in degrees by multiplying it by 180.0 and dividing by π (3.14).
- If `flag` is `1`, the function interprets the binary string as a 16-bit floating-point number in a custom format. It extracts the sign, exponent, and fraction parts of the binary number and converts them to a floating-point value. Special cases like NaN, infinity, and zero are handled similarly to the `flag == 3` case. The resulting floating-point value is then converted to an angle in degrees in the same manner.

The function returns the converted angle in degrees. If the `flag` argument does not match either `1` or `3`, the function returns `0`.

This function is useful for converting binary data representing angles into a human-readable format, which can be particularly useful in applications dealing with sensor data or other measurements that require angle representation.

2) binToFloatMag

```
double binToFloatMag
(
    std::string binaryStr,
    int flag
)
```

The `binToFloatMag` function takes two arguments:

- `std::string binaryStr` : A string representing a binary number.
- `int flag` : An integer flag that determines how the binary number should be interpreted and converted.

The function converts a binary number to a floating-point number representing a magnitude. The conversion process depends on the value of the `flag` argument:

- If `flag` is `3`, the function interprets the binary string as a 32-bit floating-point number in IEEE 754 format. It extracts the sign, exponent, and fraction parts of the binary number and converts them to a floating-point value. Special cases like NaN (Not a Number), infinity, and zero are handled appropriately. The resulting floating-point value represents the magnitude.
- If `flag` is `1`, the function interprets the binary string as a 16-bit floating-point number in a custom format. It extracts the sign, exponent, and fraction parts of the binary number and converts them to a floating-point value. Special cases like NaN, infinity, and zero are handled similarly to the `flag == 3` case. The resulting floating-point value represents the magnitude.

The function returns the converted magnitude as a floating-point number. If the `flag` argument does not match either `1` or `3`, the function returns `0.0`.

This function is useful for converting binary data representing magnitudes into a human-readable format, which can be particularly useful in applications dealing with sensor data or other measurements that require magnitude representation.

Other helper functions used

```
bool closeOrNot
(
    bool &isClose, std::mutex &mtx
)

std::string removeSubstring
(
    std::string str
```

```

)

std::string returnHex
(
char buffer[], int start, int end
)

std::string hexValue
(
char buffer[], int start, int end
)

std::string removeSpaces
(
std::string str
)

int binaryToDecimal
(
std::string binaryString
)

std::string removeTrailingZeros
(
std::string str
)

std::string hexToAscii
(
std::string hex
)

std::string toHex
(
std::string data
)

int hexToDecimal
(
std::string str
)

std::string hexToBinary

```

```
(  
    std::string hex  
)
```

closeOrNot

- **Arguments:**
- `bool &isClose` : A reference to a boolean variable indicating whether a condition is met.
- `std::mutex &mtx` : A reference to a mutex object used for thread synchronization.
- **Functionality:** This function locks the mutex, checks the value of `isClose`, unlocks the mutex, and returns the value of `isClose`. It's used to safely check a condition that might be shared across multiple threads.

removeSubstring

- **Arguments:**
- `std::string str` : The input string from which a substring will be removed.
- **Functionality:** This function removes all occurrences of the substring "ffffff" from the input string and returns the modified string.

returnHex

- **Arguments:**
- `char buffer[]` : A character array containing the data to be converted.
- `int start` : The starting index of the portion of the array to convert.
- `int end` : The ending index of the portion of the array to convert.
- **Functionality:** Converts a portion of a character array to a hexadecimal string.

hexValue

- **Arguments:**
- `char buffer[]` : A character array containing the data to be converted.
- `int start` : The starting index of the portion of the array to convert.
- `int end` : The ending index of the portion of the array to convert.
- **Functionality:** Converts a portion of a character array to a hexadecimal string and removes all occurrences of the substring "ffffff" from the hexadecimal string.

removeSpaces

- **Arguments:**

- `std::string str` : The input string from which spaces will be removed.
- **Functionality:** Removes all spaces from the input string and returns the modified string.

binaryToDecimal

- **Arguments:**
- `std::string binaryString` : A string representing a binary number.
- **Functionality:** Converts a binary number represented as a string to its decimal equivalent.

removeTrailingZeros

- **Arguments:**
- `std::string str` : The input string from which trailing zeros will be removed.
- **Functionality:** Removes trailing zeros from the input string and returns the modified string.

hexToAscii

- **Arguments:**
- `std::string hex` : A string representing hexadecimal data.
- **Functionality:** Converts a hexadecimal string to its ASCII representation.

toHex

- **Arguments:**
- `std::string data` : The input string to be converted to hexadecimal.
- **Functionality:** Converts a string to its hexadecimal representation.

hexToDecimal

- **Arguments:**
- `std::string str` : A string representing a hexadecimal number.
- **Functionality:** Converts a hexadecimal number represented as a string to its decimal equivalent.

hexToBinary

- **Arguments:**
- `std::string hex` : A string representing a hexadecimal number.
- **Functionality:** Converts a hexadecimal number represented as a string to its binary representation.

These functions collectively provide a set of tools for manipulating and converting strings, which are useful in converting the obtained data into human readable data.

Contributions

The software was built by a team of third year students from IIT Ropar in 2024.

Contributors for the backend portion of the application are as follows

1. Arpit Kumar Verma (2021eeb1156@iitrpr.ac.in)
2. Arpit Vats (2021eeb1157@iitrpr.ac.in)
3. Priyanshu Nanda (2021eeb1197@iitrpr.ac.in)
4. Rahul (2021eeb1199@iitrpr.ac.in)

ACKNOWLEDGEMENT

We express profound gratitude for the invaluable support received from our esteemed supervisor, Dr. Bibhu Prasad Padhy. His unwavering commitment and dedication to our project have been nothing short of remarkable. Throughout the entirety of our project journey, Dr. Padhy has been an indispensable guide, offering insightful direction and mentorship that has been pivotal to our progress. His guidance, coupled with our consistent and dedicated meetings, has been instrumental in navigating the complexities of the project's development process. Moreover, the constant feedback provided by Dr. Padhy has been both constructive and enlightening, significantly enhancing our understanding and approach to the project.

In addition to the personal mentorship we received from Dr. Padhy, we also wish to extend our appreciation to the Indian Institute of Technology (IIT) Ropar. Their support has been a cornerstone in enabling us to bring this project to fruition. The opportunity to undertake this project under the umbrella of IIT Ropar has not only allowed us to apply our academic knowledge in a practical setting but has also enriched our learning experience immeasurably. The project has served as a platform for us to grow both professionally and personally, equipping us with invaluable skills and insights that will undoubtedly be beneficial in our future endeavors.

Reflecting on our journey, we are deeply appreciative of the opportunity to learn and grow through this project. The experience has been transformative, offering us a unique perspective on the challenges and rewards of project-based learning. We are grateful for the support and guidance provided by Dr. Padhy and IIT Ropar, which have been instrumental in shaping our project's success and our own personal and professional development.

Table of Contents	
1.	User Guide For Installation and Use and System Requirements
2.	Code Explanation 1. Header Files 2. Function Definitions 3. Class Definition 4. Algorithm Flowchart 5. Working of Code => Title Window => Adding Number of PMUs => Login and PMU Details => Displaying PMU Information and Disconnect Button => Displaying PMU Window => Plotting Voltage Graphs with ImPlot => Plotting Current Graphs with ImPlot => Plotting Frequency Graphs with ImPlot
3.	Guide for Making Changes in Original Code and Set Up Requirements
4.	Conclusion

User Guide For Installation and Use and System Requirements

Steps to follow:

1. Go to the Github repository >>[depproject/RTPDC_software \(github.com\)](https://github.com/depproject/RTPDC_software)
2. Download the zip and extract the folder.
3. Open the file named RTPDC and you are good to go.
4. The given folder includes:
 - “Fonts”: This folder contains the font styles used in the application.
 - “RTPDC”: This is the main application file.
 - “libcrypto-3-x64.dll , libmysql.dll, libssl-3-x64.dll”: This file contains the data in text format.
 - “ImGui.ini and RTPDC.pdb”: Other System Files.

System Requirements:

1. You may require mysql if you want to store the data in SQL.

CODE EXPLANATION

Header Files

The code includes several header files that are required for the implementation. These header files provide the necessary libraries and functions for the code to work properly. Some of the included header files are:

- backend/backend.h:** This header file is a custom header file located in the backend directory.
- imgui.h:** This header file is part of the Dear ImGui library, which is a popular GUI library for C++.
- imgui_plot.h:** This header file is part of the ImPlot library, which is an extension for Dear ImGui that provides plotting functionality.
- imgui_impl_dx9.h:** This header file is part of the Dear ImGui library and provides DirectX 9 implementation for ImGui.
- imgui_impl_win32.h:** This header file is part of the Dear ImGui library and provides Win32 implementation for ImGui.
- d3d9.h:** This header file is part of the DirectX 9 library and provides the necessary functions and structures for DirectX 9 programming.
- tchar.h:** This header file provides functions and macros for working with Unicode strings.
- imgui_impl_glfw.h:** This header file is part of the Dear ImGui library and provides GLFW implementation for ImGui.
- imgui_impl_opengl3.h:** This header file is part of the Dear ImGui library and provides OpenGL 3 implementation for ImGui.

Function Definitions

updateData Function

- This function takes an unordered map `dfmap`, an integer `k`, and a reference to an object of the `dat` class `pm` as parameters. It updates the data stored in the `pm` object based on the values in the `dfmap` map.
- The function first calculates the value of `c` by adding the `soc` value and the `fracsec` value from the `dfmap` map. It then pushes the calculated value of `c` to the `iTime` vector and the `freq` value to the `iFreq` vector in the `pm` object.
- If the `sizeassigned` variable in the `pm` object is 0, the function assigns the size of the `phasors` vector in the `dfmap` map to the `phasoraxes` vector in the `pm` object.
- The function then iterates over the `phasors` vector in the `dfmap` map and pushes the values to the `phasoraxes` vector in the `pm` object.
- If the size of the `iTime` vector in the `pm` object exceeds 100, the function removes the first element from the `iTime` vector, the first element from the `iFreq` vector, and the first two elements from each vector in the `phasoraxes` vector.
- Finally, the function sleeps for 40 milliseconds before returning.

CalculateScalingFactor Function

- This function takes two integers `viewportWidth` and `viewportHeight` as parameters and returns a float value.
- The function calculates the scaling factors `scaleX` and `scaleY` by dividing the `viewportWidth` and `viewportHeight` by the base width and height respectively.
- The function then chooses the smaller scaling factor between `scaleX` and `scaleY` and assigns it to the `scaleFactor` variable.
- Finally, the function returns the value of `scaleFactor`.

Other Helper Functions

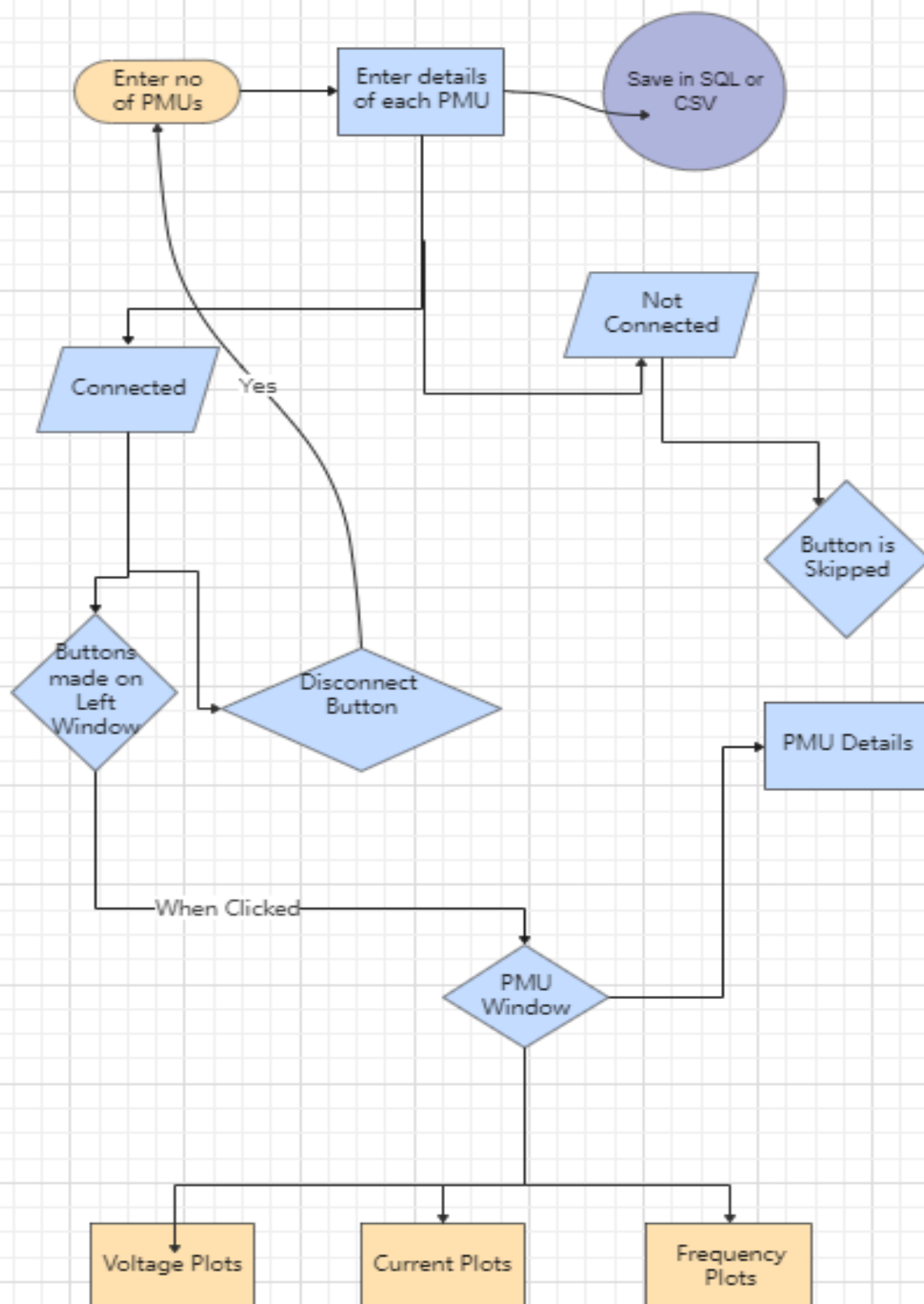
- The code also includes several other helper functions such as CreateDeviceD3D, CleanupDeviceD3D, ResetDevice, and WndProc. These functions are responsible for initializing and cleaning up the Direct3D device, handling window messages, and resetting the device when necessary.

Class Definition

The code includes a class definition called `dat`. This class has several member variables and functions. Some of the member variables are:

- `username`: A character array to store the username.
- `name`: A character array to store the name.
- `save`: A character array to store the save.
- `iport`: A character array to store the iport.
- `id`: A character array to store the id.
- `loog`: A boolean variable to store the value of loog.
- `loginSuccess`: A boolean variable to store the value of loginSuccess.
- `loginAttempted`: A boolean variable to store the value of loginAttempted.
- `fl`: A boolean variable to store the value of fl.
- `sizeassigned`: A boolean variable to store the value of size assigned.
- `iTime`: A vector of doubles to store the iTime values.
- `iFreq`: A vector of doubles to store the iFreq values.
- `phasoraxes`: A vector of vectors of doubles to store the phasoraxes values.

Algorithm Flowchart



Working of Code:

=> Title Window



Key Concepts:

ImGui::Begin() and ImGui::End(): These functions mark the beginning and end of the custom window creation. The `ImGuiWindowFlags_NoTitleBar` flag is used to remove the title bar from the window.

Margins: The `leftMargin` and `topMargin` variables are calculated based on the size of the viewport to provide spacing for the contents within the window.

Text Display: Text elements like "x" (The Project Logo rendered as font) and "R T P D C" are displayed using `ImGui::Text()` within the custom window.

Font Customization: Custom fonts (`customFont` and `customFont2`) are pushed and popped to change the font style for specific text elements.

Layout Control: Functions like `ImGui::SetCursorPosX()`, `ImGui::SetCursorPosY()`, `ImGui::SameLine()`, and `ImGui::Indent()` are used to control the layout and positioning of elements within the window.

Code Structure

The code snippet starts by creating a custom window using `ImGui::Begin()` with a specified title and flags. It then calculates margins based on the viewport size and sets the cursor position for text elements. Custom fonts are applied to specific text, and the layout is controlled using various ImGui functions. Finally, the window is closed with `ImGui::End()`.

```
ImGui::SetNextWindowPos(ImGui::GetMainViewport()->Pos);
ImGui::SetNextWindowSize(ImVec2(viewportSize.x, (viewportSize.y) / 8), ImGuiCond_Always);
ImGui::GetStyle().WindowMenuButtonPosition = ImGuiDir_None;

{
    ImGui::Begin("Your Window Title", nullptr, ImGuiWindowFlags_NoTitleBar); // Pass a pointer to the window title
    // Add contents with margins

    float leftMargin = (viewportSize.y) / 16;
    float topMargin = (viewportSize.x) / 128;

    //float scalingFactor = CalculateScalingFactor(viewportSize.x*2, viewportSize.y*2);

    ImGui::SetCursorPosX(leftMargin);
    ImGui::SetCursorPosY(topMargin * 1.2);

    //ImGuiIO& io = ImGui::GetIO();
    //io.FontGlobalScale = scalingFactor/1.5;

    ImGui::PushFont(customFont);
    ImGui::Text("x");
    ImGui::PopFont();
}
```

```

ImGui::SetCursorPosX(leftMargin);
ImGui::SetCursorPosY(topMargin * 1.2);

//ImGuiIO& io = ImGui::GetIO();
//io.FontGlobalScale = scalingFactor/1.5;

ImGui::PushFont(customFont);
ImGui::Text("x");
ImGui::PopFont();

ImGui::SetCursorPosX(leftMargin);
ImGui::SetCursorPosY(topMargin);
ImGui::SameLine();
ImGui::SetCursorPosY(topMargin * 4);
ImGui::Indent(viewportSize.x / 2.5);
ImGui::PushFont(customFont2);
ImGui::Text("R T P D C");
ImGui::PopFont();

//ImGui::Text("An application to plot real-time synchrophasors from PMUs connected via PDCs ");

ImGui::End();

```

=> Adding Number of PMUs

```

ImGui::Begin("Add Number of PMUs");

int flg = 0;
char tot[3];

if (flg == 0)
{
    ImGui::InputText(" ", tot, 3);

    if (ImGui::Button("Enter"))
    {
        // Perform validation and store the entered value
    }
}

```

This creates an ImGui window titled "Add Number of PMUs". Inside this window, the code checks if a flag flg is set to 0. If it is, the code displays an input text field for the user to enter the number of PMUs. The entered value is stored in the tot variable.

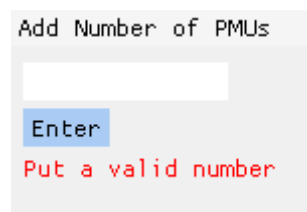
Next, the code checks if the "Enter" button is pressed. If it is, the code performs some validation on the entered value. If the entered value is not a valid number, the flg flag is set

to 1, indicating an error. Otherwise, the entered value is converted to an integer and stored in the tot_pmu variable.

If the flg flag is set to 1, indicating an error, the code displays a text message in red color saying "Put a valid number".

After that, the code checks if the cli flag is set to true. If it is, the code displays buttons for each PMU. The number of buttons displayed is equal to the value stored in the tot_pmu variable.

Finally, the code displays a "Disconnect" button. When this button is pressed, it resets various variables and flags, clears data structures, and performs other cleanup tasks.



=> Login and PMU Details

```

if (en && flg == 0 && !lcf)
{
    ImGui::Begin("Login");
    ImGui::Checkbox("Save in SQL", &csvordb);
    if (csvordb)
    {
        static char serverAddress[100] = "localhost";
        static char userName[100] = "root";
        static char password[100] = "AAPR@203";
        static char port[10] = "3306";

        ImGui::InputText("Enter Server Address", serverAddress, IM_ARRAYSIZE(serverAddress));
        ImGui::InputText("Enter Username", userName, IM_ARRAYSIZE(userName));
        ImGui::InputText("Enter password", password, IM_ARRAYSIZE(password));
        ImGui::InputText("Enter port", port, IM_ARRAYSIZE(port));
        if (ImGui::Button("Save"))
        {
            std::string a = "\0";
            mymutex.lock();
            for (int j = 0; j < 100; j++)
            {
                db.serverAddress[j] = serverAddress[j]; if (serverAddress[j] == a[0]) break;
            }
            for (int j = 0; j < 100; j++)
            {
                db.userName[j] = userName[j]; if (userName[j] == a[0]) break;
            }
            for (int j = 0; j < 100; j++)
            {
                db.password[j] = password[j]; if (password[j] == a[0]) break;
            }

            //std::string a = "\0";
            //if (pm[i].save[0] == a[0]) { pm[i].fl = 0; break; }
            //pm[i].name[j] = pm[i].save[j];

            db.port = std::stoi(port);
            mymutex.unlock();
        }
    }
}

```

```

for (int i = 1; i <= tot_pmu; i++)
{
    char sn[100]; sprintf(sn, "%d", i);
    std::string s1 = "IP Address "; s1 += sn; int x1 = s1.length(); char str1[100]; for (int i = 0; i < x1; i++) { str1[i] = s1[i]; }
    std::string s2 = "Port Number "; s2 += sn; int x2 = s2.length(); char str2[100]; for (int i = 0; i < x2; i++) { str2[i] = s2[i]; }
    std::string s3 = "PMU ID "; s3 += sn; int x3 = s3.length(); char str3[100]; for (int i = 0; i < x3; i++) { str3[i] = s3[i]; }
    ImGui::Text("Please Enter the Details");

    // Input field for username
    ImGui::InputText(str1, pm[i].username, IM_ARRAYSIZE(pm[i].username));
    ImGui::InputText(str2, pm[i].ipport, IM_ARRAYSIZE(pm[i].ipport));
    ImGui::InputText(str3, pm[i].id, IM_ARRAYSIZE(pm[i].id));
    pm[i].sizeassigned = 0;
}

if (tot_pmu) {
    if (ImGui::Button("Enter")) {
        pmuserentered = 1;
        // pmuVector.clear();
        // sockets.clear();
        for (int i = 1; i <= tot_pmu; i++) {
            if (strlen(pm[i].username) == 0 || strlen(pm[i].ipport) == 0 || strlen(pm[i].id) == 0) {
                showIncompleteDetailsMessage = true; // Set the flag to show the message
            }
            else {
                showIncompleteDetailsMessage = false;
                // Set the flag indicating a login attempt was made
            }
        }
        if (showIncompleteDetailsMessage == false) {
            for (int i = 1; i <= tot_pmu; i++) {
                mymutex.lock();
                PMU temp(pm[i].username, std::stoi(pm[i].ipport), std::stoi(pm[i].id));
                pmuVector.push_back(temp);
                mymutex.unlock();
                id_to_i[std::stoi(pm[i].id)] = i;
            }
        }
    }
}

std::thread taskThread(backend, std::ref(isClose), pmuVector.size(), pmuVector, csvordb, db, std::ref(dfmap), std::ref(sockets));
taskThread.detach();
}
}

//std::cout << cli << "hello\n";
if (showIncompleteDetailsMessage)
    ImGui::TextColored(ImVec4(1.0f, 1.0f, 0.0f, 1.0f), "Kindly fill all input fields");
ImGui::End();
}

if (cli == 0 && pmuserentered) {
    //std::this_thread::sleep_for(std::chrono::milliseconds(4000));
    for (int i = 0; i < sockets.size(); i++) {
        if (sockets[i].first > 1) {
            cli++;
        }
    }
}
}

```

The above code is structured as follows:

Login Section: The code checks if the user is logged in (en is true) and the login flag (flg) is 0. If both conditions are true, it creates a window titled "Login" using ImGui::Begin.

Inside the window, it creates a checkbox for selecting whether to save the login details in a **SQL database** or **CSV file**.

If the checkbox is selected, it creates input fields for entering the server address, username, password, and port number for the SQL. When the "Save" button is clicked, the entered details are saved in the db object.

PMU Details Section: The code then iterates over the PMUs (Phasor Measurement Units) and creates input fields for entering the IP address, port number, and PMU ID for each PMU. The entered details are stored in the pm object.

Submit Section: If there are PMUs entered, the code creates an "Enter" button. When the button is clicked, it checks if all the required input fields for each PMU are filled. If any input field is empty, it sets a flag (`showIncompleteDetailsMessage`) to display a warning message. If all input fields are filled, it creates PMU objects with the entered details and stores them in the `pmuVector`. It also starts a background task (backend) using a separate thread to handle the entered PMUs.

Warning Message: If the `showIncompleteDetailsMessage` flag is true, it displays a warning message in yellow text using `ImGui::TextColored`.

Client Count: Finally, the code checks the number of connected clients (`cli`) and increments it based on the number of active sockets.

Key Concepts

ImGui: ImGui is a library for creating GUIs in C++. It provides a simple and intuitive way to create windows, buttons, input fields, and other UI elements.

Checkbox: The `ImGui::Checkbox` function creates a checkbox UI element. In this code, it is used to create a checkbox for selecting whether to save the login details in a SQL database.

InputText: The `ImGui::InputText` function creates an input field UI element. It allows the user to enter text, which can be used to input server address, username, password, and port number.

Button: The `ImGui::Button` function creates a button UI element. It can be used to trigger an action when clicked, such as saving the entered details or submitting the form.

Text: The `ImGui::Text` function displays text in the GUI. It is used to display instructions and labels for the input fields.

TextColored: The `ImGui::TextColored` function displays colored text in the GUI. It is used to display a warning message if the user has not filled all the required input fields.

Add Number of PMUs	Login
2	Save in SQL
Enter	Please Enter the Details
	IP Address 1
	Port Number 1
	PMU ID 1
	Please Enter the Details
	IP Address 2
	Port Number 2
	PMU ID 2
	Enter

=> Displaying PMU Information and Disconnect Button

```

if (cli)
{
    int pmucount = 1;
    for (int i = 1; i <= tot_pmu; i++)
    {
        if (sockets[i - 1].first > 1) {
            char sn[100]; sprintf(sn, "%d", pmucount);
            //ImGui::Text(" % d % d", sockets[i - 1].second, id_to_i[sockets[i - 1].second]);

            std::string s = "PMU ", nu = "\0"; s += sn; int x = s.length(); char str[100]; for (int j = 0; j < x; j++) { str[j] = s[j]; }str[x]
            //if (pm[i].fl) { for (int j = 0; j < 100; j++) { str[j] = pm[i].name[j]; } } //is it needed??

            ImGui::PushItemWidth(ImGui::GetFontSize() * -12);
            ImGui::PushID(i);
            if (id_to_i[sockets[i - 1].second] == show) {
                ImGui::PushStyleColor(ImGuiCol_Button, (ImVec4)ImColor::HSV(0.775f, 0.65f, 0.99f));
                ImGui::PushStyleColor(ImGuiCol_ButtonHovered, (ImVec4)ImColor::HSV(0.775f, 0.65f, 0.99f));
            }
            else {
                ImGui::PushStyleColor(ImGuiCol_Button, (ImVec4)ImColor::HSV(0.566f, 0.42f, 0.99f));
                ImGui::PushStyleColor(ImGuiCol_ButtonHovered, (ImVec4)ImColor::HSV(i / 7.0f, 0.7f, 0.7f));
            }

            ImGui::PushStyleColor(ImGuiCol_ButtonActive, (ImVec4)ImColor::HSV(i / 7.0f, 0.8f, 0.8f));
            if (ImGui::Button(str, ImVec2((viewportSize.x) / 9, (viewportSize.y) * 1 / 32))) {
                show = id_to_i[sockets[i - 1].second]; buttonflg = 1;
            }

            ImGui::Text("%d", sockets[i - 1].first);
            ImGui::PopStyleColor(3);
            ImGui::PopID();

            pmucount++;
        }
    }
}

```



```

}
if (ImGui::Button("Disconnect", ImVec2((viewportSize.x) / 9, (viewportSize.y) * 1 / 32))) {
    mymutex.lock();
    isClose = 1;
    mymutex.unlock();
    pm.erase(pm.begin(), pm.end());
    dfmap.clear();
    sockets.clear();
    pmuVector.clear();
    cfgMap.clear();
    sidebar = 0;
    cli = 0;
    id_to_i.clear();

    //
    // Main loop

    tot_pmu = 0; flg = 0; en = 0; lgf = 0;
    memset(tot, '\0', sizeof(tot));
    pmusertered = 0;
    csvordb = 0;
    show = -1;
    isEnabled = 0;

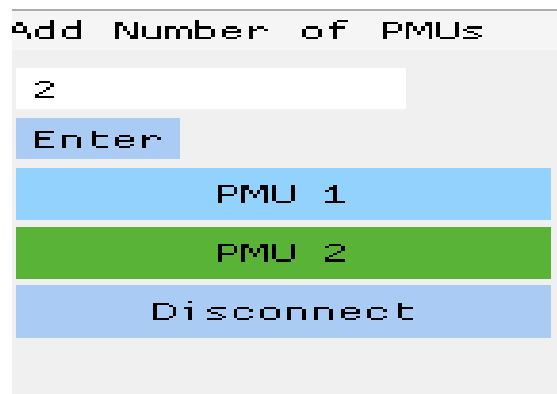
    loog = true; buttonflg = 0;
    showIncompleteDetailsMessage = false;
    showInvalidIDsMessage = false;
}

ImGui::End();

```

The code above is structured as follows:

- The code starts with a conditional statement **if (cli)** to check if the variable *cli* is true.
- Inside the conditional statement, there is a loop that iterates over the PMUs.
- Within the loop, there are several ImGui functions used to display PMU buttons for only the connected PMUs and to implement the formatting options of the buttons like changing colors on hovering, selecting, etc.
- When a button for a PMU is clicked the value of **show** is set to the **i** corresponding to the PMU. This helps in displaying the window of the PMU clicked the code for which is explained later
- After the loop, there is another ImGui button for disconnecting from the PMUs. It sets the value of all the desired components to their initial value and restarts the program.



=> Displaying PMU Window

```
if (show != -1)
{
    lgf = 1;
    std::string a = "0";
    char sn[100]; sprintf(sn, "%d", show);
    ImGui::Begin(" 2nd", nullptr, ImGuiWindowFlags_NoTitleBar);    // Pass a pointer to our bool variable (the window

    ImGui::Text("Details of the PMU Connected");

    ImGui::Text("Ip Address:  ");
    ImGui::SameLine();
    ImGui::Text("%s", (pm[show].username));

    ImGui::Text("Port Address:  ");
    ImGui::SameLine();
    ImGui::Text("%s", pm[show].iport);

    ImGui::Text("PMU ID:  ");
    ImGui::SameLine();
    ImGui::Text("%d", std::stoi(pm[show].id));

    updateData(dfmap, std::stoi(pm[show].id), pm[show]);
    //if (buttonflg) { std::this_thread::sleep_for(std::chrono::milliseconds(4000)); buttonflg = 0; }
    std::vector<std::string> o = cfgMap[std::stoi(pm[show].id)].chnam; int n = o.size(); std::string nu = " \0";

    ImGui::BeginTabBar("MyTabBar");
}
```

The above code snippet is executed when a button is clicked and show is changed to i from -1.

On clicking a PMU button:

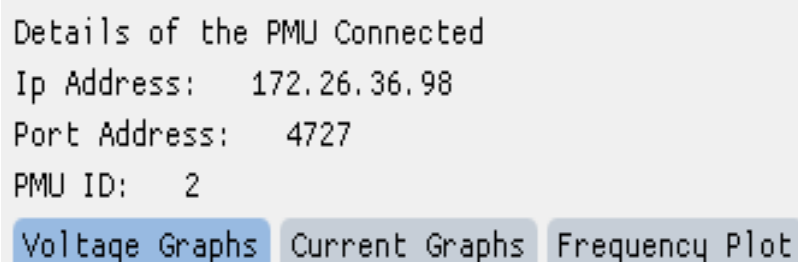
Initialization: The variable lgf is set to 1, indicating the start of a new ImGui window.

ImGui Window Creation: A new ImGui window titled "2nd" is created without a title bar using ImGui::Begin.

Display PMU Details: The code displays the IP Address, Port Address, and PMU ID of the PMU indexed by the show variable.

The **updateData(dfmap, std::stoi(pm[show].id), pm[show])** is called which updates the graph points of the clicked PMU data upto 100 points and deletes the previous points.

A tab bar is also created to implement the different plots.



The screenshot shows a window titled "Details of the PMU Connected". It displays the following information: Ip Address: 172.26.36.98, Port Address: 4727, and PMU ID: 2. Below this information is a tab bar with three tabs: "Voltage Graphs", "Current Graphs", and "Frequency Plot". The "Voltage Graphs" tab is currently selected and highlighted in blue.

=> Plotting Voltage Graphs with ImPlot:

```
{
    if (ImGui::BeginTabItem("Voltage Graphs"))
    {
        // ImGui::Text("%d", std::stoi(pm[show].id));
        //ImGui::Text("%d", dfmap[std::stoi(pm[show].id)].phasors.size());
        //ImGui::Text("%d", pm[show].phasoraxes.size());
        // ImGui::Text("%.4f", pm[show].phasoraxes[1][0]);
        ImVec2 availableSpace = ImGui::GetWindowSize();

        ImPlotStyle& style = ImPlot::GetStyle();
        // Calculate the width for each plot to fit them side by side responsively
        float plotWidth = (availableSpace.x - ImGui::GetStyle().ItemSpacing.x) * 0.49f;
        float plotheight = (availableSpace.y - ImGui::GetStyle().ItemSpacing.y) / 3.8f;

        for (int i = 0; i < dfmap[std::stoi(pm[show].id)].phasors.size(); i++) {
            if (o[i].find("R") != std::string::npos)
            {
                ImPlot::SetNextLineStyle(ImVec4(1.0f, 0.0f, 0.0f, 1.0f)); // Red Line Plot color
            }
            if (o[i].find("Y") != std::string::npos)
            {
                ImPlot::SetNextLineStyle(ImVec4(0.8f, 0.7f, 0.0f, 1.0f)); // Yellow Line Plot color
            }
            if (o[i].find("B") != std::string::npos)
            {
                ImPlot::SetNextLineStyle(ImVec4(0.0f, 0.0f, 1.0f, 1.0f)); // Blue Line Plot color
            }
            ImPlot::StyleColorsLight(); // Increase line width
            style.PlotBorderSize = 1.0f; // Set the thickness of the border
            style.Colors[ImPlotCol_PlotBorder] = ImVec4(0.0f, 0.0f, 0.0f, 1.0f); // To give border color

            int l = o[i].length(); char str[100]; for (int j = 0; j < l; j++) { str[j] = o[i][j]; }str[l] = nu[0]; str[l + 1] = nu[0];
            for (int j = 0; j < 100; j++) { str[l + 1 + j] = sn[j]; if (sn[j] == nu[1]) break; }

            if (o[i].find("VOLTAGE") != std::string::npos)
            {
                if (ImPlot::BeginPlot(str, ImVec2(plotWidth, plotheight))) {

                    if (o[i].find("VOLTAGE") != std::string::npos)
                    {
                        if (ImPlot::BeginPlot(str, ImVec2(plotWidth, plotheight))) {

                            ImPlot::SetupAxisFormat(ImAxis_X1, "%Y-%m-%d %H+5:%M+30:%S"); // Use a date-time format string
                            ImPlot::SetupAxisScale(ImAxis_X1, ImPlotScale_Time); // Indicate that the X-axis uses time data

                            ImPlot::SetupAxisLimits(ImAxis_X1, pm[show].iTime.front(), pm[show].iTime.back(), ImGuiCond_Always);
                            //ImPlot::SetupAxisLimits(ImAxis_Y1, 10, 15, ImGuiCond_Always);
                            // Plot your data
                            ImPlot::PlotLine("Voltage", pm[show].iTime.data(), pm[show].phasoraxes[2 * i].data(), 100);
                            ImPlot::EndPlot();
                        }
                        ImGui::SameLine();

                        if (ImPlot::BeginPlot("Phase Angle", ImVec2(plotWidth, plotheight))) {

                            ImPlot::SetupAxisFormat(ImAxis_X1, "%Y-%m-%d %H+5:%M+30:%S"); // Use a date-time format string
                            ImPlot::SetupAxisScale(ImAxis_X1, ImPlotScale_Time); // Indicate that the X-axis uses time data

                            ImPlot::SetupAxisLimits(ImAxis_X1, pm[show].iTime.front(), pm[show].iTime.back(), ImGuiCond_Always);
                            ImPlot::SetupAxisLimits(ImAxis_Y1, 0 - 200, 200, ImGuiCond_Always);
                            // Plot your data
                            ImPlot::PlotLine("Theta in degrees", pm[show].iTime.data(), pm[show].phasoraxes[(2 * i) + 1].data(), 100);
                            ImPlot::EndPlot();
                        }
                    }
                }
            }
        }
        ImGui::TextColored(ImVec4(1.0f, 0.0f, 0.0f, 1.0f), "To zoom in to the nearest value in graph , double click on Y-axis");
        ImGui::EndTabItem();
    }
}
```

The code provided is a part of a larger program and focuses on the section responsible for plotting voltage graphs using `ImPlot`. Let's break down the code structure and understand its functionality:

BeginTabBar: This function call starts a new tab bar in the user interface. It is used to organize different sections of the interface into tabs.

BeginTabItem: This function call starts a new tab item within the tab bar. It represents a specific tab in the user interface.

ImVec2 availableSpace = ImGui::GetWindowSize(); This line of code retrieves the available space within the current tab item. It will be used to calculate the width and height of the plots.

ImPlotStyle& style = ImPlot::GetStyle(); This line of code retrieves the current `ImPlot` style, which can be modified to customize the appearance of the plots.

for loop: This loop iterates over the voltage data and plots each voltage graph using `ImPlot`. It sets the line color based on the type of voltage (R, Y, or B) and applies some additional styling to the plots.

ImPlot::BeginPlot: This function call starts a new plot within the tab item. It takes the plot title and the desired plot size as parameters.

ImPlot::SetupAxisFormat: This function call configures the format of the X-axis labels. In this code example, it uses a date-time format string.

ImPlot::SetupAxisScale: This function call indicates that the X-axis uses time data. It ensures that the X-axis is scaled correctly for time-based data.

ImPlot::SetupAxisLimits: These function calls set the limits for the X and Y axes. They ensure that the plot displays the desired range of data.

ImPlot::PlotLine: This function call plots a line graph using the provided data points. It takes the plot label, X-axis data, Y-axis data, and the number of data points as parameters.

ImPlot::EndPlot: This function call ends the Voltage plot.

ImGui::SameLine: This function call ensures that the next plot is displayed on the same line as the previous plot.

ImGui::TextColored: This function call displays a text message in a colored font. In this code example, it provides a tip to the user about zooming in on the graph.

ImGui::EndTabItem: This function call ends the current tab item.

The above code plots the magnitude as well as phase angles received for the Voltage phases of the PMU.

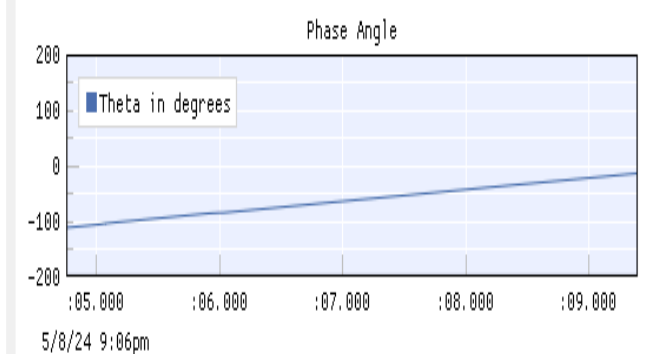
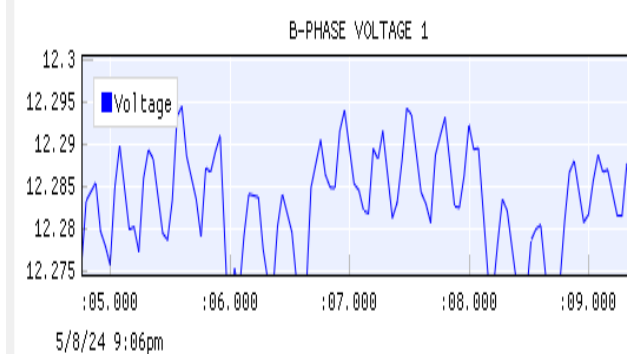
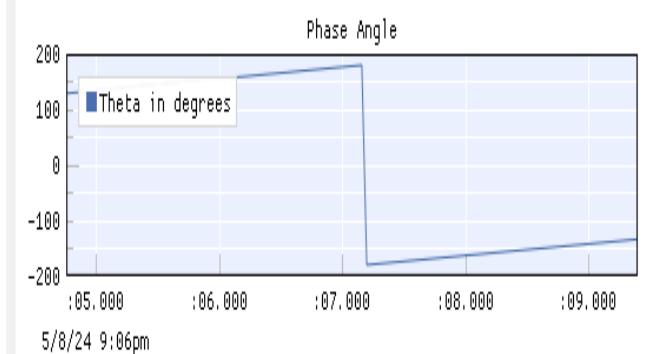
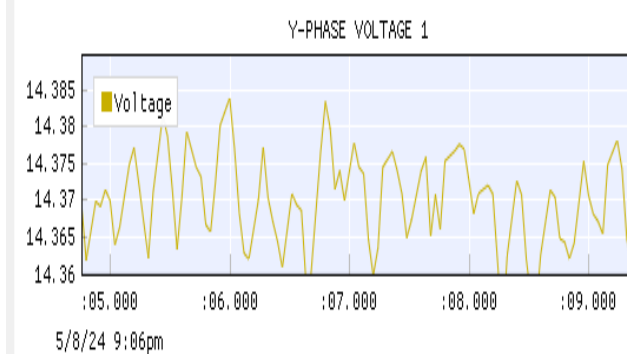
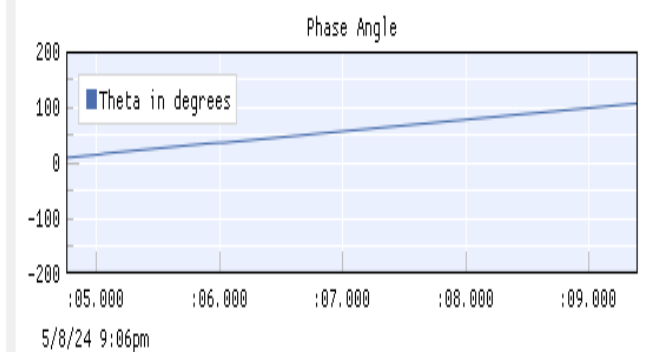
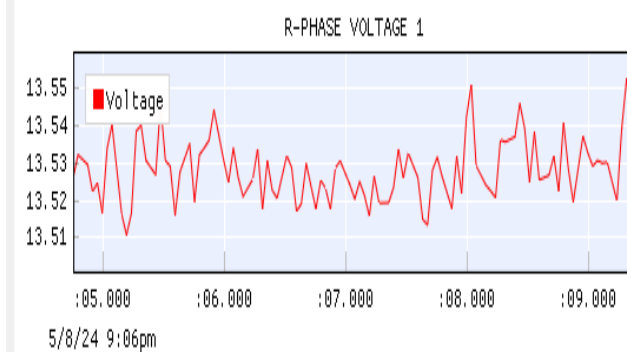
Details of the PMU Connected

Ip Address: 172.26.36.98

Port Address: 4727

PMU ID: 2

Voltage Graphs Current Graphs Frequency Plot



To zoom in to the nearest value in graph , double click on Y-axis

=> Plotting Current Graphs with ImPlot:

```
if (ImGui::BeginTabItem("Current Graphs"))
{
    //ImGui::Text("%d", dfmap[std::stoi(pm[show].id)].phasors.size());
    //ImGui::Text("%.4f", pm[show].phasoraxes[1][0]);
    ImVec2 availableSpace = ImGui::GetWindowSize();

    ImPlotStyle& style = ImPlot::GetStyle();
    // Calculate the width for each plot to fit them side by side responsively
    float plotWidth = (availableSpace.x - ImGui::GetStyle().ItemSpacing.x) * 0.49f;
    float plotheight = (availableSpace.y - ImGui::GetStyle().ItemSpacing.y) / 3.8f;

    for (int i = 0; i < dfmap[std::stoi(pm[show].id)].phasors.size(); i++) {
        if (o[i].find("R") != std::string::npos)
        {
            ImPlot::SetNextLineStyle(ImVec4(1.0f, 0.0f, 0.0f, 1.0f)); // Red plot color
        }
        if (o[i].find("Y") != std::string::npos)
        {
            ImPlot::SetNextLineStyle(ImVec4(0.8f, 0.7f, 0.0f, 1.0f)); // Yellow plot color
        }
        if (o[i].find("B") != std::string::npos)
        {
            ImPlot::SetNextLineStyle(ImVec4(0.0f, 0.0f, 1.0f, 1.0f)); // Blue plot color
        }
        ImPlot::StyleColorsLight(); // Increase line width
        style.PlotBorderSize = 1.0f; // Set the thickness of the border
        style.Colors[ImPlotCol_PlotBorder] = ImVec4(0.0f, 0.0f, 0.0f, 1.0f); // To give border color
        int l = o[i].length(); char str[100]; for (int j = 0; j < l; j++) { str[j] = o[i][j]; }str[l] = nu[0]; str[l + 1] = nu[0];
        for (int j = 0; j < 100; j++) { str[l + 1 + j] = sn[j]; if (sn[j] == nu[1]) break; }

        if (o[i].find("CURRENT") != std::string::npos)
        {
            if (ImPlot::BeginPlot(str, ImVec2(plotWidth, plotheight))) {
                ImPlot::SetupAxisFormat(ImAxis_X1, "%Y-%m-%d %H+5:%M+30:%S"); // Use a date-time format string

                ImPlot::SetupAxisScale(ImAxis_X1, ImPlotScale_Time); // Indicate that the X-axis uses time data

                ImPlot::SetupAxisLimits(ImAxis_X1, pm[show].iTime.front(), pm[show].iTime.back(), ImGuiCond_Always);
                //ImPlot::SetupAxisLimits(ImAxis_Y1, 10, 15, ImGuiCond_Always);
                // Plot your data
                ImPlot::PlotLine("Current", pm[show].iTime.data(), pm[show].phasoraxes[2 * i].data(), 100);
                ImPlot::EndPlot();
            }

            ImGui::SameLine();

            // Store old values to restore later if needed

            if (ImPlot::BeginPlot("Phase Angle", ImVec2(plotWidth, plotheight))) {
                ImPlot::SetupAxisFormat(ImAxis_X1, "%Y-%m-%d %H+5:%M+30:%S"); // Use a date-time format string
                ImPlot::SetupAxisScale(ImAxis_X1, ImPlotScale_Time); // Indicate that the X-axis uses time data

                ImPlot::SetupAxisLimits(ImAxis_X1, pm[show].iTime.front(), pm[show].iTime.back(), ImGuiCond_Always);
                ImPlot::SetupAxisLimits(ImAxis_Y1, 0 - 200, 200, ImGuiCond_Always);
                // Plot your data
                ImPlot::PlotLine("Theta in degrees", pm[show].iTime.data(), pm[show].phasoraxes[(2 * i) + 1].data(), 100);
                ImPlot::EndPlot();
            }
        }
    }

    ImGui::TextColored(ImVec4(1.0f, 0.0f, 0.0f, 1.0f), "To zoom in to the nearest value in graph , double click on Y-axis");
    ImGui::EndTabItem();
}
```

The above code is structured as follows:

BeginTabItem: This function starts a new tab for displaying the current graphs.

availableSpace: This variable stores the available space for the plots.

plotWidth and plotHeight: These variables calculate the width and height of each plot based on the available space.

Loop: The code loops through the current values and checks their type (R, Y, or B). Based on the type, it sets the plot color using the `SetNextLineStyle` function.

StyleColorsLight: This function increases the line width of the plot.

PlotBorderSize and Colors: These variables set the thickness and color of the plot border.

str: This variable stores the label for each plot.

BeginPlot: This function starts a new plot for the current or phase angle values.

SetupAxisFormat and SetupAxisScale: These functions set the format and scale of the X-axis (time).

SetupAxisLimits: These functions set the limits of the X and Y axes.

PlotLine: This function plots the current or phase angle values on the plot.

EndPlot: This function ends the current plot.

SameLine: This function aligns the phase angle plot next to the current plot.

TextColored: This function displays a message in red color to instruct the user to double-click on the Y-axis to zoom in.

EndTabItem: This function ends the tab for the current graphs.

The above code plots the magnitude as well as phase angles received for the Current phases of the PMU.

Details of the PMU Connected

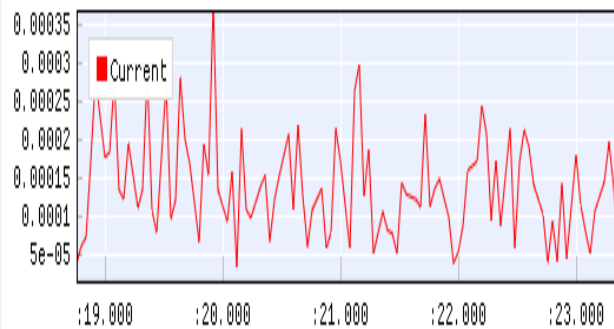
Ip Address: 172.26.36.98

Port Address: 4727

PMU ID: 2

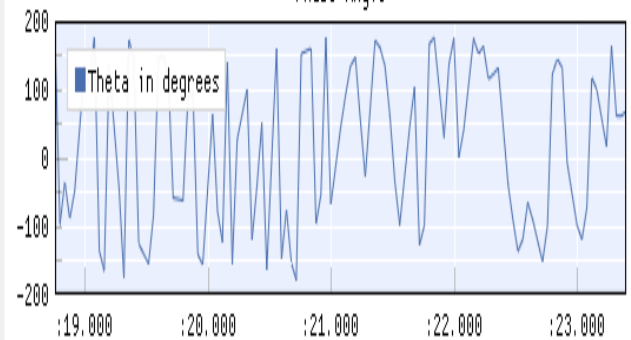
Voltage Graphs Current Graphs Frequency Plot

R-PHASE CURRENT 1



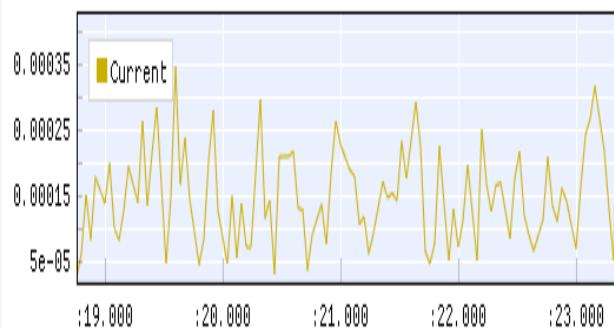
5/8/24 9:05pm

Phase Angle



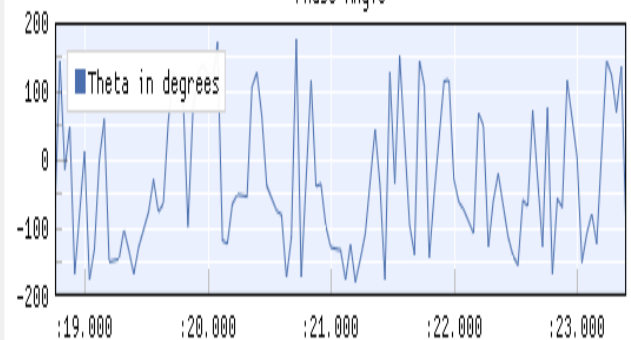
5/8/24 9:05pm

Y-PHASE CURRENT 1



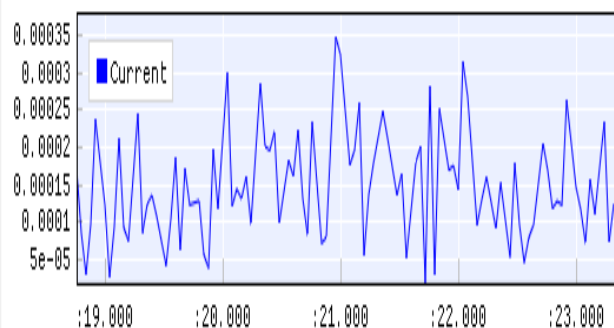
5/8/24 9:05pm

Phase Angle



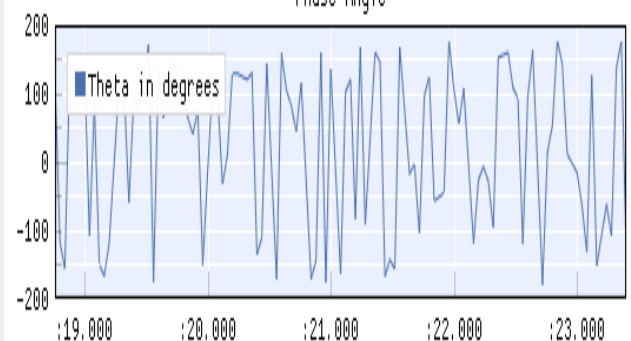
5/8/24 9:05pm

B-PHASE CURRENT 1



5/8/24 9:05pm

Phase Angle



5/8/24 9:05pm

=> Plotting Frequency Graphs with ImPlot:

```
if (ImGui::BeginTabItem("Frequency Plot"))
{
    ImVec2 availableSpace = ImGui::GetWindowSize();

    // Calculate the width for each plot to fit them side by side responsively
    // Retrieve current style
    ImPlotStyle& style = ImPlot::GetStyle();

    // Store old values to restore later if needed
    float oldBorderSize = style.PlotBorderSize;
    ImVec4 oldBorderColor = style.Colors[ImPlotCol_PlotBorder];

    // Set new border size and color
    style.PlotBorderSize = 1.0f; // Set the thickness of the border
    style.Colors[ImPlotCol_PlotBorder] = ImVec4(0.0f, 0.0f, 0.0f, 1.0f); // Black border

    // Calculate plot dimensions
    float plotWidth = (availableSpace.x - ImGui::GetStyle().ItemSpacing.x) * 1.0f;
    float plotHeight = (availableSpace.y - ImGui::GetStyle().ItemSpacing.y) / 1.2f;

    // Begin the plot with specified dimensions
    if (ImPlot::BeginPlot("Frequency", ImVec2(plotWidth, plotHeight))) {

        ImPlot::SetupAxisFormat(ImAxis_X1, "%Y-%m-%d %H:%M:%S"); // Use a date-time format string
        ImPlot::SetupAxisScale(ImAxis_X1, ImPlotScale_Time); // Indicate that the X-axis uses time data

        // Setup axis limits
        ImPlot::SetupAxisLimits(ImAxis_X1, pm[show].iTime.front(), pm[show].iTime.back(), ImGuiCond_Always);

        // Plot data
        ImPlot::PlotLine("Frequency", pm[show].iTime.data(), pm[show].iFreq.data(), 100);

        // End the plot
        ImPlot::EndPlot();
    }

    // Restore previous style settings if needed elsewhere
    style.PlotBorderSize = oldBorderSize;
}
```

```
style.Colors[ImPlotCol_PlotBorder] = oldBorderColor;
ImGui::TextColored(ImVec4(1.0f, 0.0f, 0.0f, 1.0f), "To zoom in to the nearest value in graph , double click on Y-axis");
ImGui::EndTabItem();
}

ImGui::EndTabBar();

}

ImGui::End();
}
```

The code snippet provided is a part of a larger codebase and is enclosed within an if statement that checks if the "Frequency Plot" tab is active. It uses ImGui functions to create a tab item and sets up the necessary plot dimensions and styles using ImPlot functions. It then begins the plot using ImPlot::BeginPlot and plots the frequency data using ImPlot::PlotLine. Finally, it ends the plot using ImPlot::EndPlot and restores the previous style settings.

The provided code snippet demonstrates how to create a frequency plot using the ImGui and ImPlot libraries in C++. It sets up the necessary plot dimensions, styles, and data, and then plots the frequency data over time.



- We have converted UNIX time to UTC time by using pre built function `SetAxisFormat(ImAxis_X1, "%Y-%m-%d %H:%M:%S")`

Guide for Making Changes in Original Code and Set Up Requirements

1. Setup Visual Studio on your device. <https://visualstudio.microsoft.com/>
2. Setup ImGui in your device. <https://github.com/ocornut/imgui>
3. Setup Implot in your device. <https://github.com/epezent/implot>
4. Create a folder named RTPDC >> Cppwithgui and then include folders 'fonts' , 'backend' and 'vendor'.
5. Include the backend code in the folder backend.
6. Include two folders 'iimgui' and 'implot' in 'vulkan' and include all the extracted files of ImGui & Implot extracted above in the above two folders respectively.
7. Then create the 'Source.cpp' which is the main file in which the executable code is written.
8. Now create the solution format file of 'Cppwithgui' in visual studio.

You have to follow the above steps if you want to begin with scratch and make your own solution and hence a working application

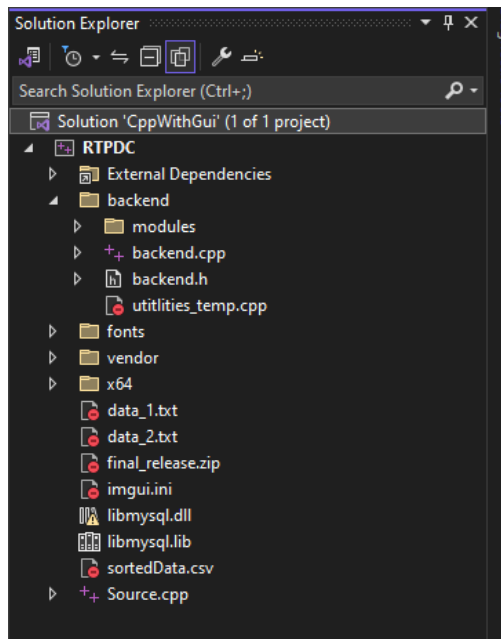
OR

You can download our Solution directly from drive link . [RTPDC - Google Drive](#)

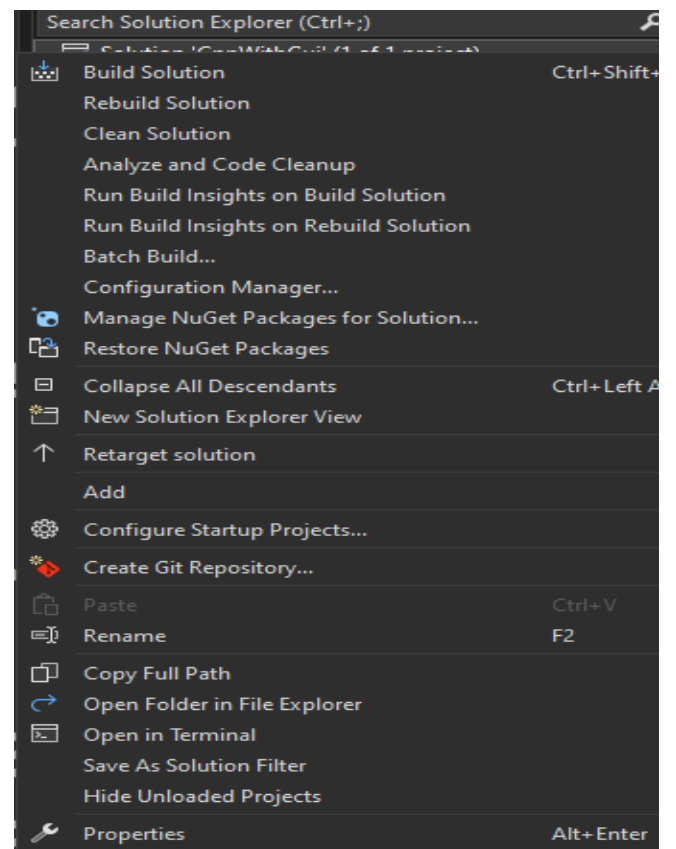
In both scenarios you have to follow further steps mentioned below.

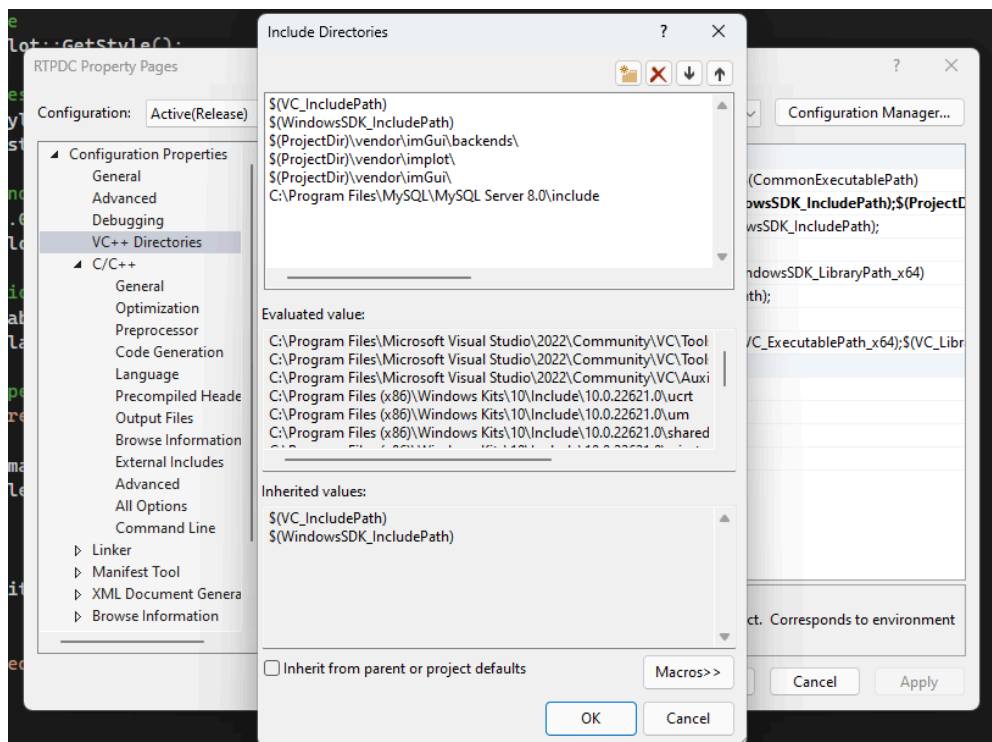
9. The most crucial part is setting up the properties of the solution.

Your Solution explorer will be looking like this by now

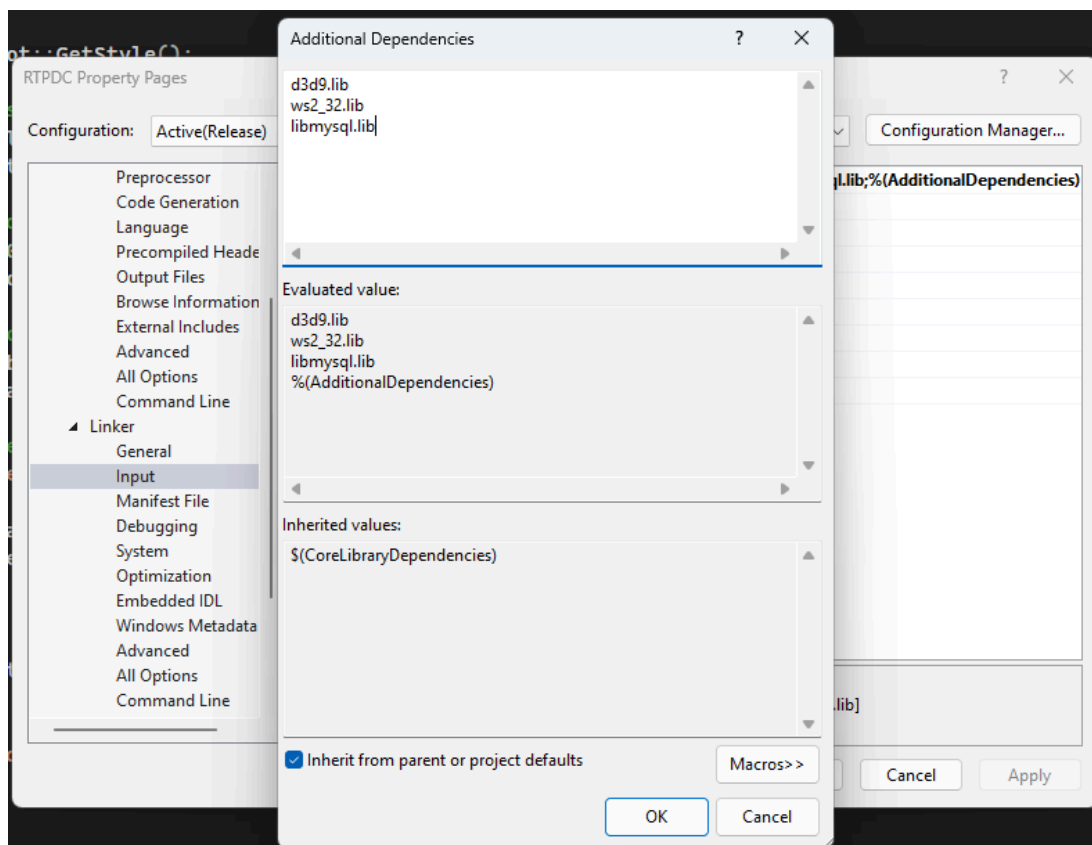


Right Click on RTPDC >> Click Properties



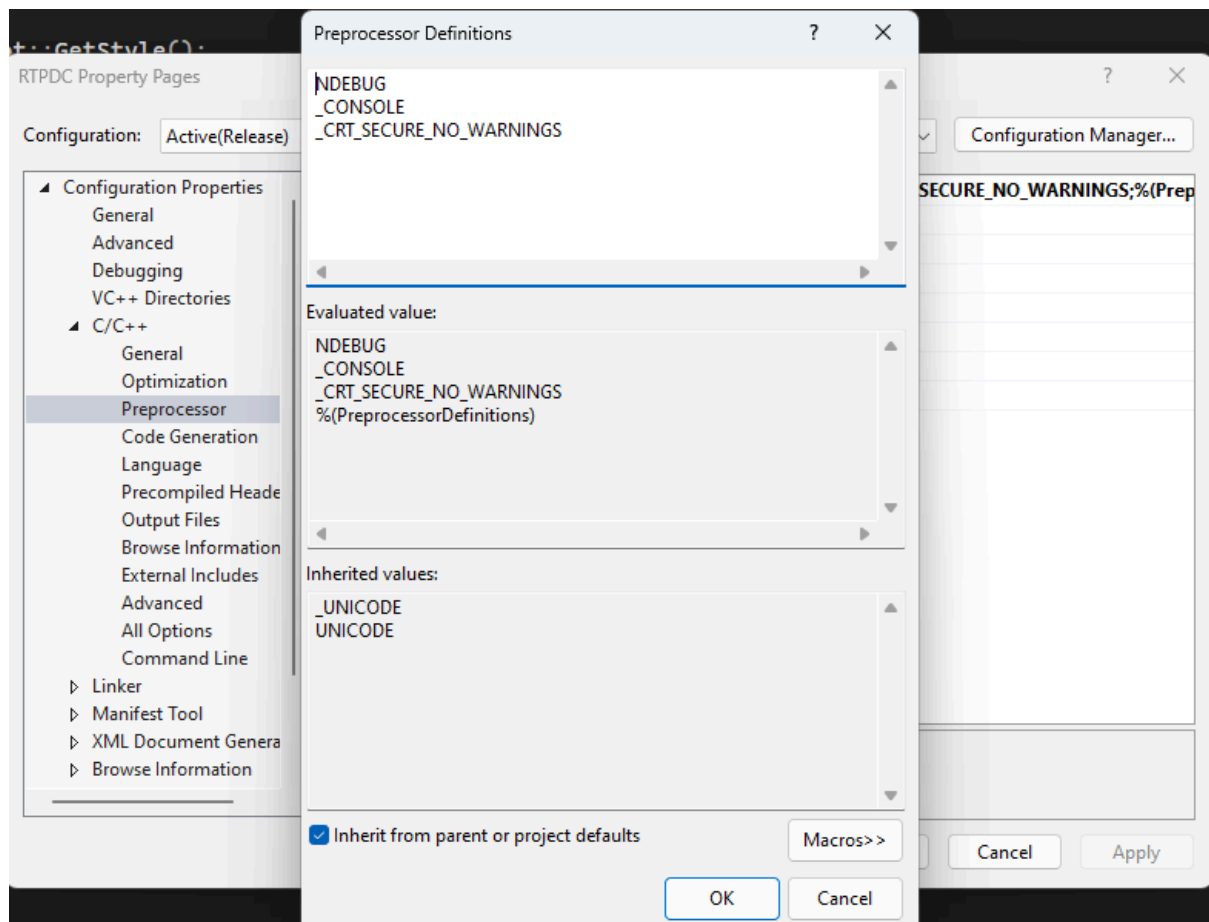


Include all above in VC++ Directories



Include all above in Linker>>Input Properties

Include all below in C/C++ >> Preprocessor Directories



10. You are all set to go. Just go to the source file, make required changes and run in Debug mode during building the Project and run.
11. After all the required Changes you can release the application by converting Debug to Release mode and rewriting all the properties required again in mentioned Configurational properties.

Conclusion:

1. Objective Fulfillment:

- The primary objective of our project was to design a frontend interface for real-time visualization of voltage and current values for all three phases using ImGui and ImPlot.
- We successfully achieved this goal by developing a user-friendly interface that displays the real-time data in a visually appealing manner.

2. Technology Integration:

- By leveraging ImGui and ImPlot libraries enabled us to create a dynamic and interactive frontend interface.
- ImGui provided a flexible and efficient GUI framework, while ImPlot facilitated the plotting of real-time data with ease.

3. Real-Time Data Visualization:

- Our frontend interface offers real-time plotting of voltage and current values for all three phases.
- Users can visualize fluctuations and trends in the electrical parameters over time, enabling quick analysis and decision-making.
- The graphical representation of data enhances understanding and facilitates effective monitoring of power systems.

4. Scalability and Customization:

- Our frontend design is scalable and customizable, allowing for future expansion and additional features.
- As the project evolves, new functionalities can be integrated seamlessly into the existing interface without significant modifications.

5. Multiple PMU's:

- Our project excels in displaying plots of multiple PMUs simultaneously.
- By using advanced technologies and efficient data handling, our frontend interface seamlessly accommodates the visualization of data from multiple PMUs concurrently.